

Machine Learning for High Dimensional Data Analysis

Master's Thesis

Sasi Tummala
Matriculation number: 7376921

February 26, 2026

First Supervisor: Dr. Guido Meier
Second Supervisor: Prof. Hans Fangohr

Max Planck Institute for the Structure and Dynamics of Matter

Contents

1	Introduction	5
1.1	Micromagnetics	5
1.2	Supervised and Unsupervised learning	6
2	Conceptual Framework	8
2.1	Micromagnetic energy terms	8
2.2	Curse of Dimensionality	9
2.3	Introduction to Autoencoders	13
2.4	Clustering Algorithms	16
2.4.1	K-means Clustering	16
2.4.2	Density-Based Spatial Clustering of Applications with Noise (DB-SCAN)	18
2.4.3	Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN)	19
2.4.4	Gaussian Mixture Model	27
2.5	Evaluation Metrics for Clustering	29
2.5.1	Internal Metrics	29
2.5.2	External Metrics	30

2.6	Variational Autoencoder	32
2.6.1	Variational Inference	34
2.6.2	Kullback-Leibler Divergence	35
2.6.3	ELBO Function	36
2.6.4	ELBO optimization and the Reparametrization trick	37
2.6.5	Posterior Collapse	38
3	Methodology	40
3.1	Tools used	40
3.2	Data	40
3.2.1	Dataset 1	41
3.2.2	Dataset 2	42
3.3	Autoencoder Implementation	43
3.3.1	Convolutional Block	44
3.3.2	Linear Block	44
3.3.3	Flatten and Global Pooling	45
3.3.4	Final Tanh layer	45
3.4	Network architecture for Dataset 1	46
3.5	Network architecture for Dataset 2	47
3.6	Variational Autoencoder	48
3.6.1	Latent Space Parameterization	48
3.6.2	Reparameterization Trick	49
3.6.3	Loss Function and Regularization	50
3.6.4	Latent Representation Extraction	50

3.7	Mitigating Posterior Collapse	50
3.8	Training Procedure	51
3.9	Hyperparameter Optimization	52
4	Results and Outlook	54
4.1	Dataset 1	54
4.1.1	Model Reconstruction Performance	54
4.1.2	Clustering Evaluation	60
4.1.3	Latent Space Structure	61
4.1.4	Generative Exploration of the Latent Space	62
4.2	Dataset 2	63
4.2.1	Model Reconstruction Performance	63
4.2.2	Clustering Evaluation	71
4.2.3	Latent Space Structure	74
4.2.4	Generative Exploration of the Latent Space	76
4.3	Outlook	78
4.4	Software Availability	79

Abstract

High-dimensional scientific data often contain rich structures that are difficult to analyze using conventional methods due to the curse of dimensionality. This thesis investigates the use of machine learning—specifically autoencoders and clustering techniques—for unsupervised analysis of high-dimensional micromagnetic simulation data. The study focuses on learning compact latent representations of magnetization configurations and identifying physically meaningful patterns without relying on labeled data.

Micromagnetic states are represented as high-dimensional vector fields, making direct clustering unreliable. To address this, convolutional autoencoders and variational autoencoders are employed to perform nonlinear dimensionality reduction while preserving essential physical features. The learned latent spaces are subsequently analyzed using multiple clustering algorithms, including K-means, DBSCAN, HDBSCAN, and Gaussian Mixture Models. Clustering quality is evaluated using both internal and external metrics to assess cohesion, separation, and agreement with known structures.

Overall, the work shows that combining deep representation learning with unsupervised clustering offers an effective framework for discovering patterns in complex physical systems. The approach reduces computational complexity, eliminates the need for manual annotation, and enables systematic exploration of large simulation datasets.

Chapter 1

Introduction

1.1 Micromagnetics

Ferromagnetic materials are described by different theories at different length scales. At the atomic scale, individual electron spins interact through quantum-mechanical exchange, producing a strong local alignment. At the scale of 10 – 100 nm, collective spin behavior forms domains and complex internal structures. The computational cost of atomic simulations makes them impractical for larger geometries. At macroscopic scales, a material can often be treated as a uniform magnetic body with an average magnetization[17]. Micromagnetics bridges the atomic and macroscopic descriptions. It is very useful at the micron scale as it captures the relevant physics while making key assumptions and approximations to avoid computational intractability.

Consequently, micromagnetics has become a standard tool for investigating mesoscale magnetic phenomena in thin films, nanostructures, and patterned devices [6, 17]. Beyond fundamental research, micromagnetic modeling plays a central role in modern technological applications. It is widely used in the design of magnetic storage devices, spintronic components, magnetic sensors, and permanent magnets, where performance depends critically on domain structures and switching behavior. The framework is also essential for understanding emergent phenomena such as magnetic skyrmions, which are promising can-

didates for next-generation memory and logic technologies due to their stability and low current-driven mobility[12]. In addition, micromagnetics finds applications in biomedical engineering (e.g., magnetic nanoparticles for imaging and hyperthermia), geophysics (modeling remanent magnetization in rocks), and materials science, where it helps bridge experimental observations with theoretical predictions across multiple length scales[8].

The increasing scale and complexity of micromagnetic simulations have created a strong need for automated analysis techniques. Large parameter sweeps over geometry, material constants, temperature, and external fields can generate thousands of high-dimensional magnetization configurations, making manual inspection impractical, time-consuming, and potentially subjective. Machine learning (ML), particularly unsupervised learning, offers a systematic way to extract patterns, classify states, and identify phase transitions without requiring explicit labeling by domain experts. By learning directly from data, ML methods can reduce human intervention, improve reproducibility, and enable high-throughput exploration of parameter spaces that would otherwise be infeasible to analyze manually.

1.2 Supervised and Unsupervised learning

A fundamental criterion for classifying Machine learning models is based on the notion of supervision. Consider a quintessential Machine Learning task of categorizing a set of objects into different groups. There are two common approaches to training a model to perform this task. The first method involves choosing a representative subset of objects and manually assigning them to predefined categories. Subsequently, the model is trained to recognize objects from different groups using the labeled objects as references. This method of solving this task falls under the Supervised learning paradigm and is called Classification.

But manually labeling a dataset could prove to be a tedious process for large or complex datasets. Moreover, the number of groups and types of groups is not always obvious at the beginning of the task for some datasets. Here, An Unsupervised learning approach, called Clustering, proves to be superior. Unsupervised learning methods operate on un-

labeled data, meaning that no predefined categories or target outputs are provided during training. Instead, the model seeks to identify intrinsic patterns, structures, or regularities within the data itself.

The goal of clustering is to partition a dataset into groups of data instances with similar properties, called clusters, based on a mathematical metric of similarity. There are a multitude of ways to measure this similarity between the instances, and there are several different algorithms that are well-suited for different kinds of datasets. For the purposes of this project, and more generally, a dataset can be understood mathematically as a collection of points embedded in an abstract feature space, whose dimensionality corresponds to the number of attributes used to describe each instance. Each data instance can be represented as a vector in this high-dimensional space. It follows that geometric relationships among these vectors are common metrics used to gauge the aforementioned similarity. A discussion of some such algorithms will follow in section 2.4.

The remainder of this thesis is organized as follows. Chapter 2 begins with a brief discussion of energy terms in micromagnetics, followed by clustering methodologies, the curse of dimensionality, and the basics of autoencoders. This background establishes the theoretical motivation for applying machine learning techniques to high-dimensional magnetic data. Relevant evaluation metrics, and the theory of variational autoencoders, including variational inference and the associated optimization objectives, are presented.

Chapter 3 outlines the methodology of the study. It describes the simulation datasets, preprocessing steps, neural network architectures, training procedures, and strategies used to obtain meaningful latent representations. The implementation details of both standard autoencoders and variational autoencoders are discussed, along with techniques to mitigate common issues such as posterior collapse.

Finally, Chapter 4 presents the results of the analysis, including reconstruction performance, clustering outcomes, and the structure of the learned latent spaces for the different datasets. The thesis concludes with a discussion of the implications of these findings and potential directions for future work.

Chapter 2

Conceptual Framework

2.1 Micromagnetic energy terms

Micromagnetics is a continuum theory that models how magnetization behaves inside ferromagnetic materials at sub-micron scales. The magnetization vector field $M(r)$ is treated as a classical field throughout the material, but its magnitude and key energy terms — especially the exchange interaction — arise from the underlying quantum-mechanical spin alignment.

1. Exchange Energy :

$$A \int_V (\nabla m)^2 dV$$

Here, A is the exchange constant. This term is isotropic and favors uniform magnetization configurations.

2. Anisotropy Energy :

$$\int_V F_{\text{anisotropy}}(m) dV$$

Here, $F_{\text{anisotropy}}$ is the anisotropy energy density. This term favors configurations aligned along a preferred axis known as the easy axis.

3. **Zeeman Energy :**

$$-\mu_0 M_s \int_V m \cdot H dV$$

Here, H is the applied field, and μ_0 is the vacuum permeability. This term favors magnetization configurations that are parallel to the applied field.

4. **Demagnetization Energy :**

$$-\frac{\mu_0}{2} M_s \int_V m \cdot H_d dV$$

Here H_d is the demagnetizing field. This term favors magnetic configurations that minimize magnetic charges.

5. **Dzyaloshinskii–Moriya Interaction(DMI) Energy :**

$$\int_V Dm \cdot (\nabla \times m)$$

Here D is the spiralization tensor. This term favors magnetic configurations that are spatially rotating and have a defined chirality.

The integral is taken over the volume of the sample for all the energy terms. It describes the spatial distribution and dynamics of the magnetization vector by balancing key energy contributions — exchange, anisotropy, magnetostatic, and Zeeman energies.

2.2 Curse of Dimensionality

The Curse of Dimensionality (CoD) is a phenomenon in data analysis where an excessive number of dimensions can be detrimental, contrary to the general intuition that more information is better for analysis. The difficulties that arise due to the CoD can be explained by considering these aspects :

Data sparsity

As the dimensionality of the dataset increases, the "volume" occupied by samples grows rapidly, leading to an increased likelihood that any given neighborhood in the space contains no data, making the dataset sparse. The increased sparsity makes it difficult to perform tasks such as clustering, classification, and prediction, which rely on distance or density estimates.

A related illustration arises from uniform sampling of an n -dimensional hypercube. To maintain a fixed spatial resolution—i.e., a comparable number of samples within any local region — the number of required data points grows exponentially with dimensionality. Specifically, if k samples are placed along each axis, the total number of samples is k^n . For large n , any realistically sized dataset represents only a vanishingly small fraction of the space, implying that most regions contain no observations. Thus, without an exponential increase in sample size, high-dimensional datasets inevitably exhibit sparsity. Figure 2.1a illustrates the number of points required to uniformly sample an n -dimensional space while maintaining a fixed resolution of 5% along each dimension. The required sample size increases exponentially with dimension.

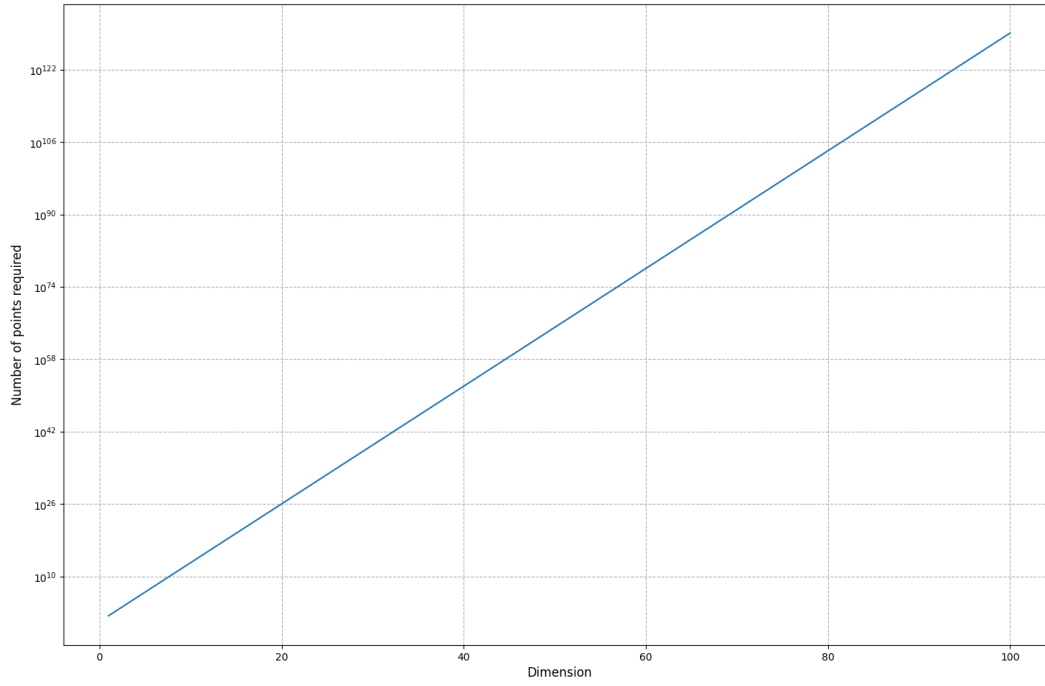
Uninformative Distance and Similarity Measures

Clustering often relies on measuring the distance or similarity between data instances. However, as the dimensionality increases, the dissimilarity between instances also increases because the mean distance between instances grows proportionally to the square root of the dimensionality. This makes it difficult to meaningfully assess the similarity or dissimilarity between subjects, which is fundamental to clustering algorithms.

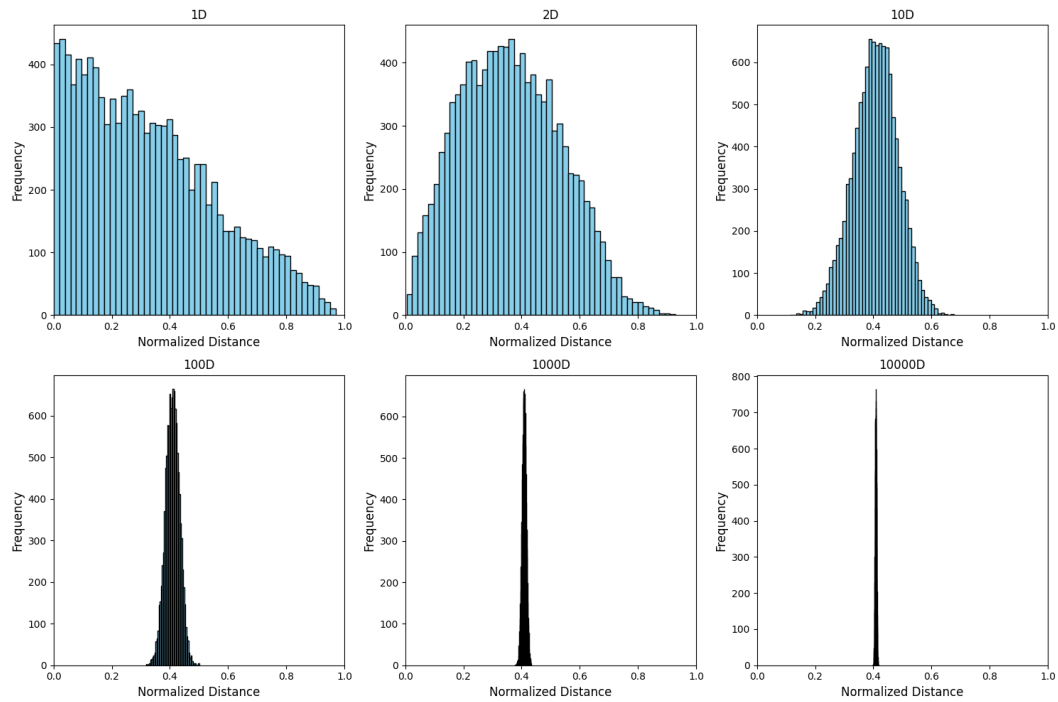
As dimensionality increases, the ratio between the distances of the nearest and farthest neighbors approaches 1[5]. This implies that all pairwise distances tend to become indistinguishable. Moreover, if all distances are nearly equal, then the distance loses its discriminatory power. This phenomenon is called the **distance concentration effect**[1].

Figure 2.1b shows that as dimensionality increases, the normalized pairwise distances between 150 random points become increasingly concentrated around a single value. Nor-

malized pairwise distance refers to the distance computed between every pair of data points, normalized to remove the effect of scale. In low dimensions, distances vary widely, producing broad distributions. At moderate dimensions, the distribution becomes more symmetric and centered. In high dimensions, the distribution sharply peaks, indicating that most distances are nearly identical.



(a) Points required to uniformly sample space (5% per dimension)



(b) Normalized Pairwise Distance Distributions in Different Dimensions

Difficulty in Outlier Assessment

Assessment of whether a particular data value is an outlier also becomes difficult in high dimensions. In one dimension, 68% of normally distributed instances fall within one standard deviation σ of the mean. However, this fraction drops drastically for large dimensionality. For example, in 10 dimensions, only 0.017% of instances fall within σ of the mean, meaning instances within σ in 10 dimensions are as rare as instances outside of 3.8σ in one dimension[35]. This can confuse clustering algorithms that might mistake normal high-dimensional instances for outliers due to their perceived rarity.

To mitigate the effects of the curse of dimensionality, **Dimensionality reduction** is crucial. However, Simple linear techniques such as PCA can only capture variance along linear directions and fail when the underlying structure is nonlinear or highly complex. More heuristic methods often discard information without learning meaningful representations[28]. Moreover, it is worth noting that these methods may also be affected by the CoD[2]. Moreover, it is worth noting that these methods are only effective

2.3 Introduction to Autoencoders

An autoencoder (AE) is a type of artificial neural network that learns to recreate the input as its output as closely as possible while passing through a lower-dimensional bottleneck layer that produces an encoding. It can be thought of as two networks working in tandem: an encoder and a decoder. An encoder compresses the input data into a lower-dimensional representation, while a decoder reconstructs the original data from this compressed representation. The AE is trained to minimize the reconstruction error between the original input and the reconstructed output. The goal is for the lower-dimensional encoding to retain as much information from the input as possible.

Formally, the original and the reconstructed space in which the instances of the dataset lie is defined as $\mathcal{X} = \mathbb{R}^n$, called the **feature space**, and the encoded space as $\mathcal{Z} = \mathbb{R}^m$, called the **latent space**. Typically, $m < n$.

- Encoder, f_θ : A function that takes $x \in \mathcal{X}$ as input and maps it to a latent vector,

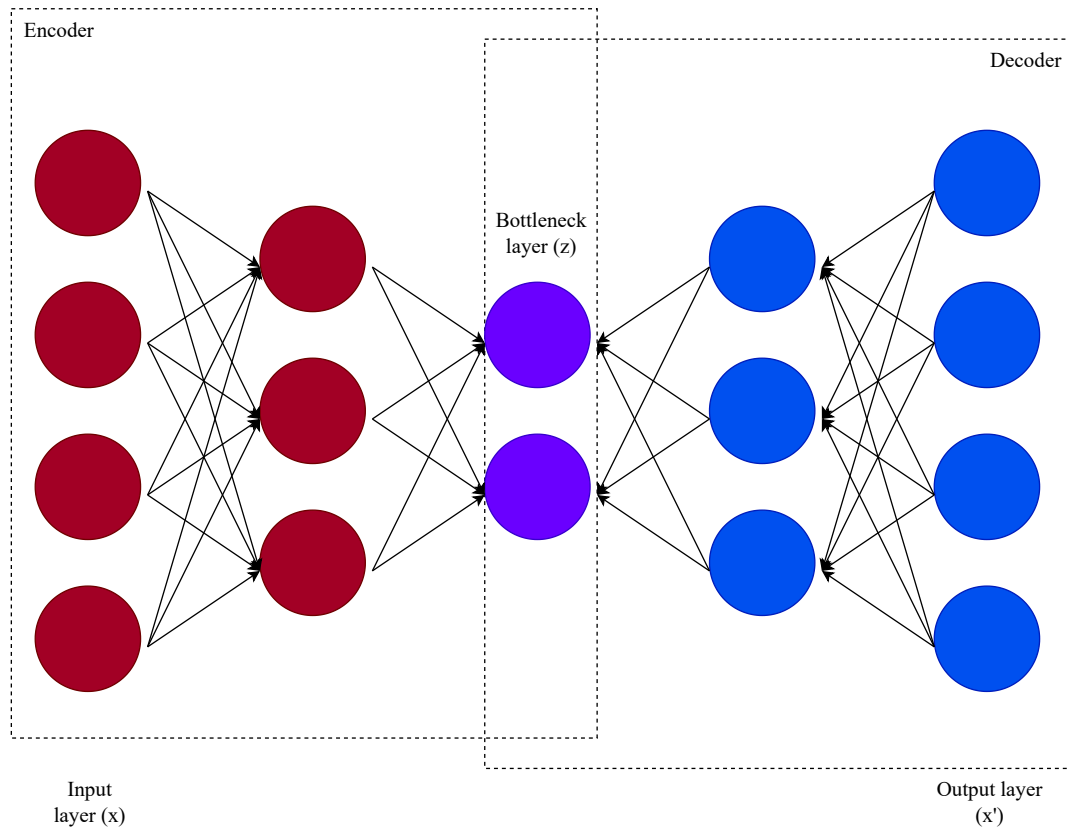


Figure 2.2: Schema of an Autoencoder

$z \in \mathcal{Z}$, parameterized by θ .

$$z = f_{\theta}(x)$$

- Decoder, g_{ϕ} : A function that maps the latent vector z back to a vector x' in the feature space, parameterized by ϕ .

$$x' = g_{\phi}(z) = g_{\phi}(f_{\theta}(x))$$

θ and ϕ are the sets of all trainable parameters (weights and biases) of the encoder and decoder networks.

A good choice for the **reconstruction loss**, that quantifies the difference between the input x and the reconstructed output x' , is the mean squared error (MSE) loss.

$$\mathcal{L}(\theta, \phi) = \mathbf{E}_{x \sim p(x)} [\|x - g_{\phi}(f_{\theta}(x))\|^2]$$

where $p(x)$ is the data distribution over the input space.

In practice, the expectation is calculated empirically using a dataset $\{x^{(i)}\}_{i=1}^N$ sampled from $p(x)$:

$$\mathcal{L}(\theta, \phi) = \frac{1}{N} \sum_{i=1}^N \|x^{(i)} - g_{\phi}(f_{\theta}(x^{(i)}))\|^2$$

The concept of an AE was initially introduced in 1986[29]. AEs were initially developed for dimensionality reduction and feature learning[15]. They remain a popular approach today. Since their introduction, they have been adapted to support a wide range of tasks, such as denoising[33], anomaly detection[30], and generative modeling [24].

And for this use case, AEs are particularly appealing because they are trained on an unlabeled dataset. Using an AE in combination with a suitable clustering algorithm completely avoids the need for manual annotation. The minimal need for human involvement makes this a highly practical option.

2.4 Clustering Algorithms

Clustering algorithms can be broadly classified into the following categories:

- Centroid-based algorithms : The aim is to minimize the distance between each instance and its assigned centroid. K-means clustering is a well-known example.
- Density-based algorithms : Algorithms such as DBSCAN or HDBSCAN identify clusters based on the density of instances in the dataset.
- Model-based: These algorithms assume a statistical model for the data and estimate the parameters of the model, for example, using Gaussian Mixture Models.

2.4.1 K-means Clustering

The K-means algorithm partitions the data into K distinct clusters such that the inertia of the clustering is minimized.

Inertia (or within-cluster sum of squares, WCSS) is a measure of the cohesiveness of the clusters. It is defined as:

$$\text{Inertia} = \sum_{i=1}^n \min_{\mu_j \in C} (\|x_i - \mu_j\|^2)$$

where:

- x_i = a data instance,
- μ_j = centroid of cluster j ,
- C = set of all centroids.

In simple terms, inertia is the sum of the squared distances of each instance to its nearest centroid. The algorithm tries to minimize inertia, leading to tighter, more compact clusters.

Following are the steps involved in performing K-means clustering:

1. **Choice of the number of clusters, K :** The optimal choice of K is essential to arrive at a meaningful clustering, but is ambiguous in some cases. The Elbow method is one technique that helps pick a suitable value for K [13]. This method involves plotting inertia against K to pick the elbow of the curve as the number of clusters.
2. **Random initialization of K cluster centroids :** K-means clustering is sensitive to the choice of initial centroids. Poor initialization can lead to getting stuck in a local optimum. K-Means++ is an initialization algorithm that mitigates this by choosing initial centroids that are more spread out[3]. It selects the first centroid randomly and each subsequent centroid with probability proportional to the squared distance from the nearest existing centroid. The algorithm can also be run several times with different initial centroids.
3. **Assignment step :** For each data instance, the distance to each centroid is computed and the data instance is assigned to the nearest centroid. As the name suggests, K-means requires a distance metric that supports centroids. Euclidean distance is the obvious and predominant choice. In order to support alternate metrics, there are modified algorithms. K-medians works well with the Manhattan metric[22]. K-medoids can work well with any arbitrary metric[27].
4. **Update step :** For each cluster, the centroid is recomputed as the mean of all assigned data instances.
5. **Convergence :** Steps 3 and 4 are repeated until the centroids do not change or the maximum number of iterations set is reached.
6. Final centroids and cluster assignments for all data instances are returned.

K-means assumes clusters are convex and isotropic data. This makes it sensitive to irregularly shaped or unevenly sized clusters. It performs poorly with categorical data or highly skewed distributions[16].

2.4.2 Density-Based Spatial Clustering of Applications with Noise (DBSCAN)

DBSCAN groups together instances that are closely packed together, i.e., instances with many nearby neighbors. instances that lie alone in low-density regions are marked as outliers.

Some definitions and parameters relevant to this discussion are :

- ϵ : The radius around a point that defines the neighborhood.
- `min_pts` : Minimum number of samples required to call a region dense.
- Core point : A point with at least `min_pts` points in its neighborhood.
- Border point : Not a core point, but in the neighborhood of one.
- Outlier point : Neither a core point, nor a border point.
- Direct-reachability : A point is directly reachable from a core point if it is within the ϵ -neighborhood of the core point. Points are only said to be directly reachable from core points. Note that this makes it non-symmetric.
- Reachability : A point is said to be reachable from a core point if there exists a chain of core points connecting the points such that every point is reachable from the adjacent point in the chain.
- Density-Connectedness : Two points are said to be density-connected if there exists a third point from which both points are reachable. Density-connectedness is symmetric.

A simple implementation of the algorithm would be the following :

1. Identify all the core points by counting the number of points in the ϵ -neighborhood of every point in the dataset.

2. Ignoring non-core points for now, assign all the core points to clusters such that all the points in a cluster are mutually density-connected.
3. Assign the Border points to the clusters if they're in the ϵ -neighborhood of one or more core points in the cluster.
4. The unassigned points at the end are the outliers.

DBSCAN has a number of advantages over k-means. The number of clusters does not have to be specified a priori. Due to the general nature of density-based clustering, DBSCAN can find arbitrarily shaped clusters. A cluster could theoretically be completely surrounded by another. It also is robust to noise, due to the inclusion of outliers.

A minor drawback is the fact that if a border point is in the neighborhood of two different clusters, the order of the data decides the cluster assignment. This lack of absolute determinism rarely creates a problem in practice[31].

A more tricky problem is to find an appropriate `min_pts` - ϵ combination. In datasets where there are large differences in densities, no combination results in an optimal clustering.

2.4.3 Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN)

HDBSCAN is an extension of DBSCAN that introduces hierarchical clustering. It retains the features of DBSCAN, such as outlier identification, but does away with a global neighborhood threshold (ϵ). Instead, it adapts to local variations in density. It constructs a hierarchy of clusters based on varying density levels and extracts the most stable clusters from this hierarchy.

Metric for Density estimation

HDBSCAN uses a different mechanism to estimate density from DBSCAN to accommodate varying densities in the dataset. Let's formally define some terms to understand this

mechanism.

Core distance : For a point x in the dataset, given a positive integer k , the core distance, denoted as $\text{core}_k(x)$, is the distance to the k th nearest neighbor of x .

Mutual reachability distance : For two points a and b in the dataset,

$$\text{mreach}_{ab} = \max\{\text{core}_k(a), \text{core}_k(b), d(a, b)\}$$

where $d(a, b)$ is the metric distance between a and b . Note that this definition of reachability is symmetric, as opposed to the way it is defined for DBSCAN.

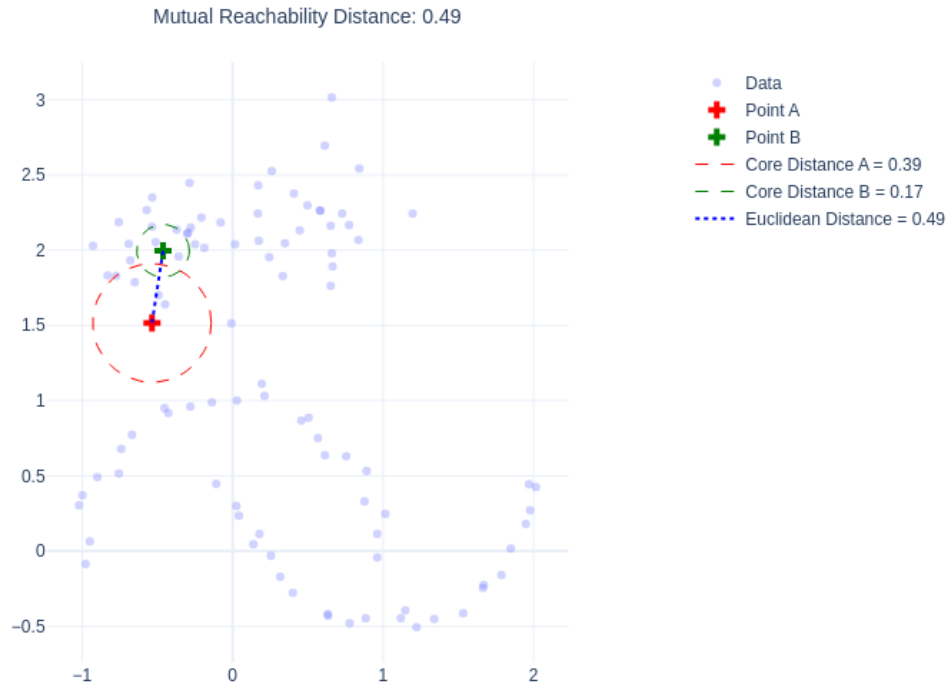


Figure 2.3: Visualization for mutual reachability distance

Minimum Spanning Tree

With the mutual reachability metric in place, we need to find clusters of dense data. But these clusters may have different densities. We shall build a weighted graph where the vertices are the points in the dataset and the weight of the edges between any two points is the mutual reachability distance between those points. From this weighted graph, we can construct a **Minimum Spanning Tree** (MST). An MST is a subset of the edges of the weighted graph that connects all the vertices together such that the sum of edge weights is as small as possible.

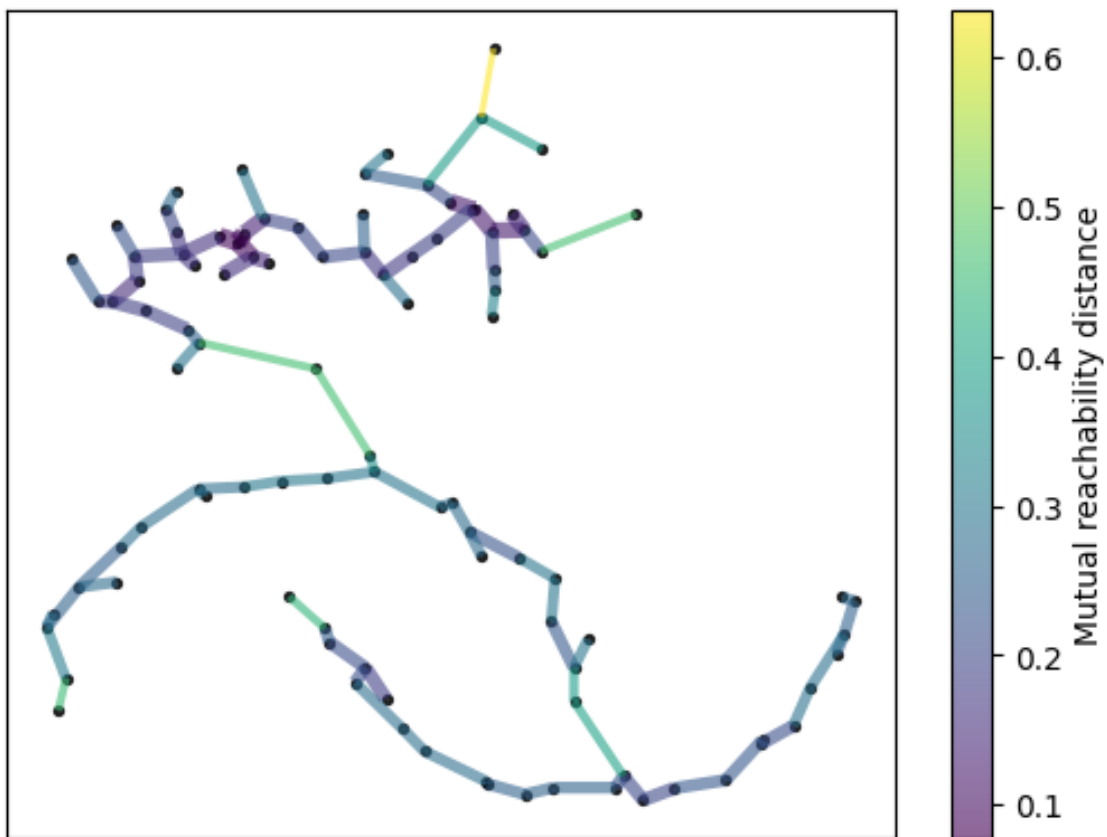


Figure 2.4: Minimum Spanning Tree (Synthetic data)

Cluster hierarchy

Now that we have the MST, we shall condense it to form a cluster hierarchy. To begin, the MST edges are sorted by increasing weight. Starting from the bottom, the edges are removed and nodes connected to the edges are merged. This leaves us with a cluster hierarchy. The edges can be interpreted as potential cluster merges and the nodes as their own clusters. An intuitive visualization of a cluster hierarchy is a **dendrogram**.

At this point, if a horizontal line is drawn through the dendrogram, the clusters it cuts through can be interpreted as the clusters at that density level. If we label any singleton clusters as noise, we've effectively performed DBSCAN clustering.

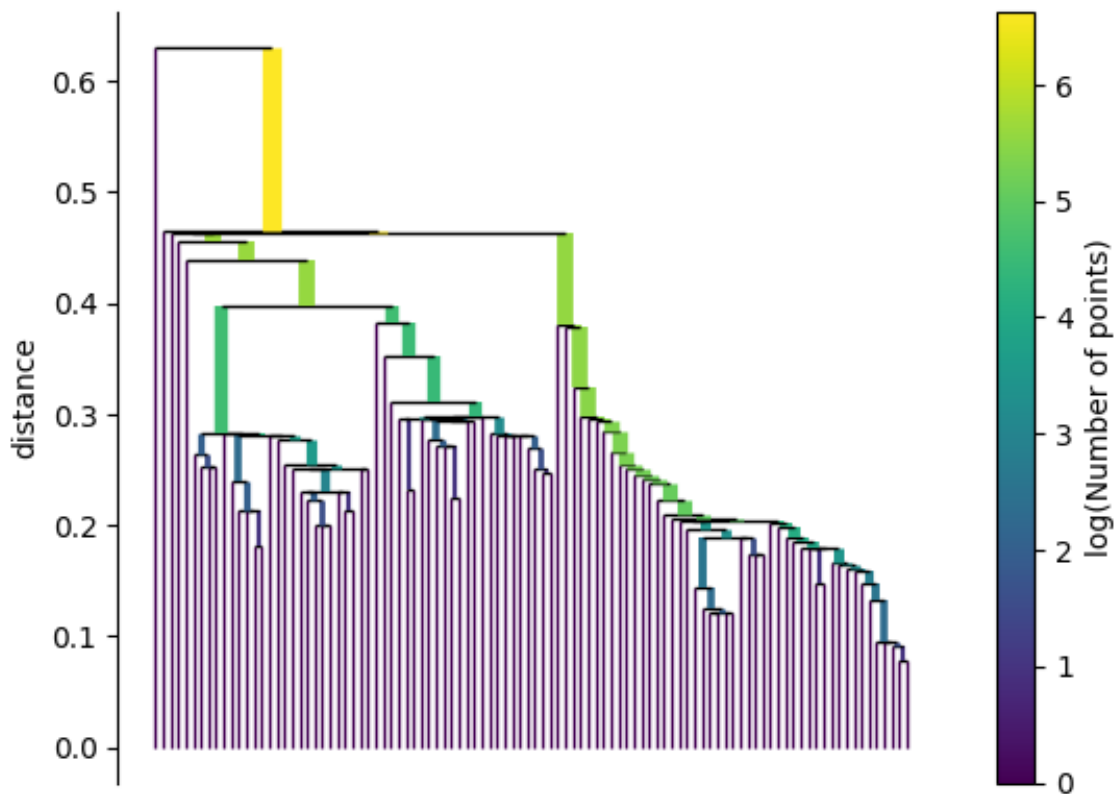


Figure 2.5: Dendrogram (Synthetic data)

Condensed Dendrogram

In order to arrive at clusters of variable density levels, let us introduce a parameter that is the lower bound for the size of the clusters, `min_cluster_size`. Now we traverse the cluster hierarchy again with this parameter in hand. At each split in the hierarchy, if a cluster has fewer points than `min_cluster_size`, they are considered as ‘falling out’ of the cluster. The larger cluster which meets the `min_cluster_size` criterion retains the parent cluster identity. If a split has two parent clusters, it persists. At the end, we arrive at a much simpler condensed tree.

This is visualized as a dendrogram with modifications. The width of the edges is proportional to the size of the cluster and the width varies as points fall out of the cluster. Intuitively, it can be said that the clusters with the largest size in the plot are more stable. They have more points in them and they persist longer through the condensation process. We need to find a way to formalize this intuition.

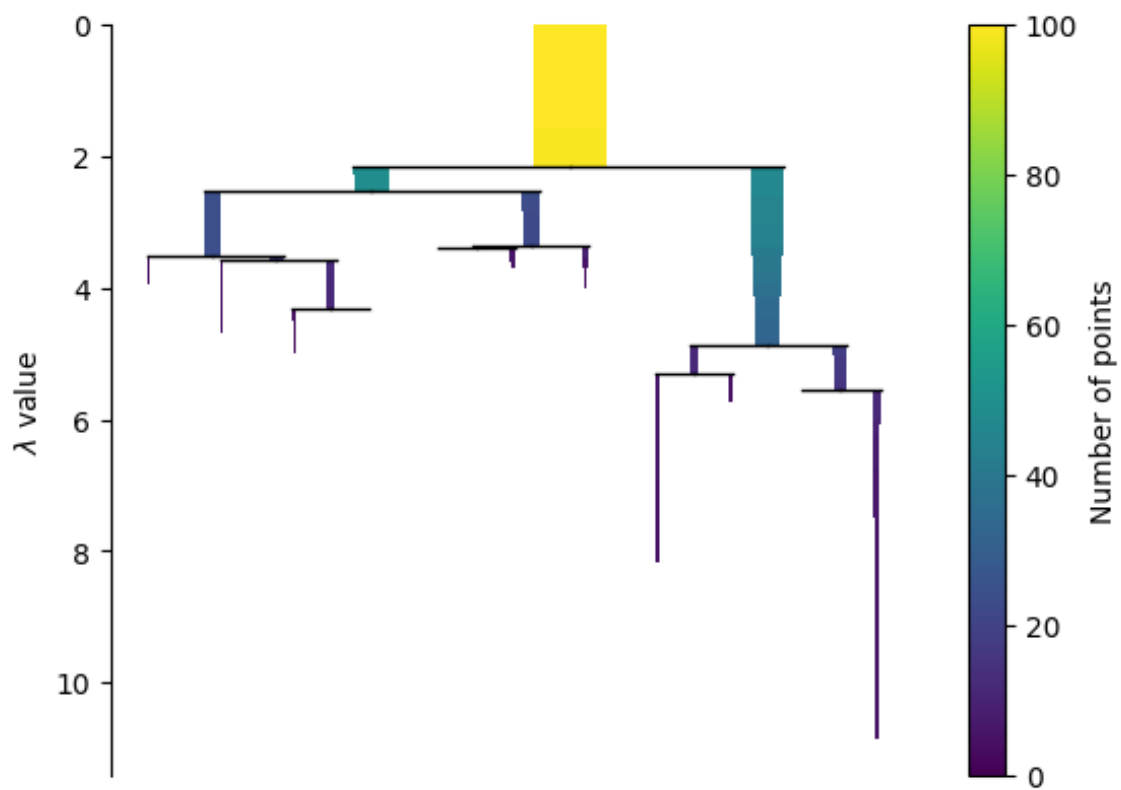


Figure 2.6: Condensed cluster hierarchy (Synthetic data)

Cluster Stability

We need a measure for the stability of the clusters. Let us define :

$$\lambda = \frac{1}{\text{distance}}$$

With this, we can calculate λ_{birth} as the λ value when the cluster is formed by splitting off and λ_{death} as the distance at which it disintegrated into smaller clusters (if it does). Every point in a cluster either falls out of a cluster somewhere between λ_{birth} and λ_{death} , or leaves the cluster when it splits. Hence, for each point p in a given cluster, we define λ_p as the value at which that point falls out.

$$\text{Stability of a cluster} = \sum_{p \in \text{cluster}} (\lambda_p - \lambda_{\text{birth}})$$

Cluster Extraction

Now that we have a way to formally compare clusters based on their stability, let us start by considering all the nodes to be clusters. We shall start at the bottom of the condensed dendrogram and work our way to the top. If a given cluster's stability is greater than the sum of the stabilities of its child clusters, the child clusters are taken out of consideration. If the opposite is true, the cluster stability is set to be the sum of the stabilities of the child clusters. Once the root node is reached, the clusters that we are left with are extracted to generate cluster labels.

HDBSCAN has all the benefits of DBSCAN mentioned in the previous section, with the added benefits of handling varying densities and having fewer parameters to optimize.

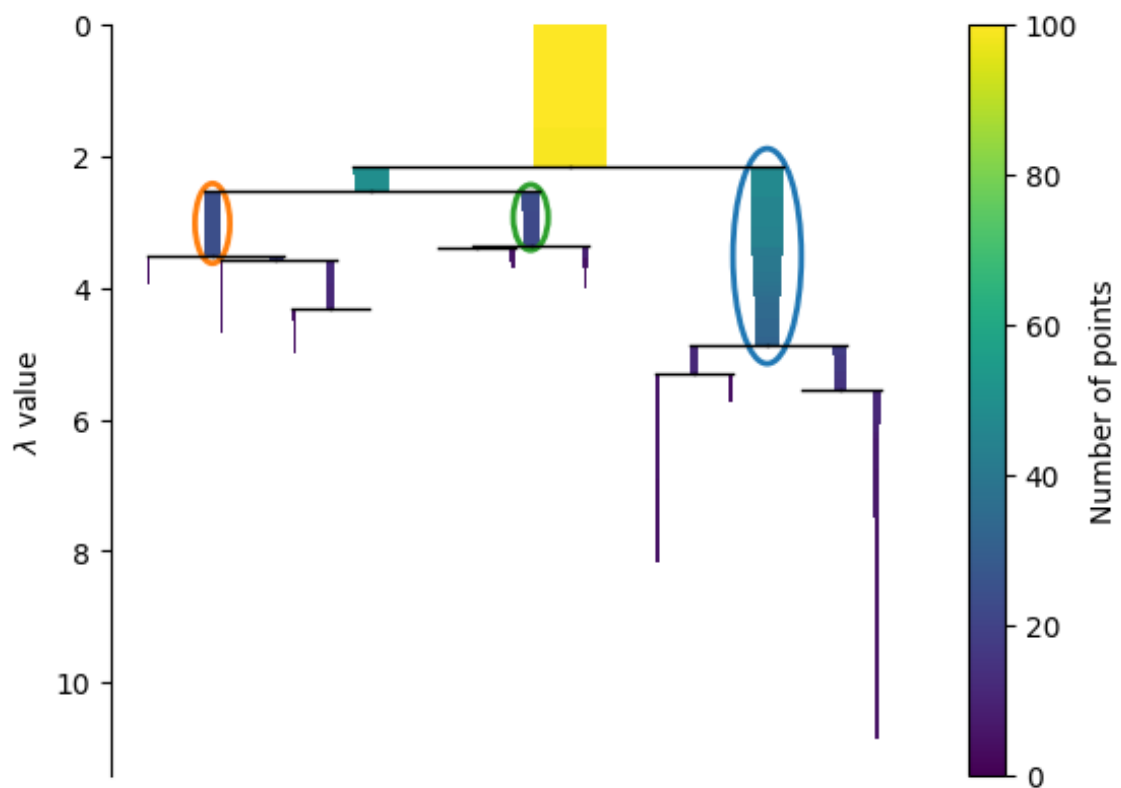


Figure 2.7: Cluster hierarchy with clusters marked (Synthetic data)

2.4.4 Gaussian Mixture Model

All the clustering techniques so far have their advantages and disadvantages. But there is one flaw that affects all algorithms which are density-based or centroid-based - the Curse of Dimensionality discussed in Chapter 1. This is where model-based algorithms, such as using a Gaussian Mixture Model (GMM), excel.

GMM clustering is a probabilistic approach to clustering that models the data as a mixture of several normal distributions. Unlike the previously discussed algorithms, which perform **hard clustering**, GMM performs **soft clustering**. That is, it assigns a probability of belonging to each cluster.

The data is assumed to come from a mixture of K Gaussian distributions, each with its own mean μ_k , covariance matrix Σ_k , and weight π_k . The probability density function of the GMM is given by:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x | \mu_k, \Sigma_k)$$

where $\mathcal{N}(x | \mu_k, \Sigma_k)$ is the Gaussian distribution with mean μ_k and covariance matrix Σ_k , and π_k is the weight of the k -th Gaussian component.

The task at hand is to find which of the K components of the GMM is most likely to have generated any given vector x_i . If z_i is the latent vector responsible for x_i ,

$$\begin{aligned} p(z_i = c | x_i) &= \frac{p(z_i = c)p(x_i | z_i = c)}{p(x_i)} \\ &= \frac{\pi_c \mathcal{N}(x_i | \mu_c, \Sigma_c)}{\sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)} \end{aligned}$$

is the posterior probability of $z_i = c$ after x_i is observed. The conditional distribution gives a soft clustering. To convert this into a hard clustering — where each x_i is assigned to just one cluster — you usually choose the most probable cluster by taking the mode of the conditional distribution, i.e., $\arg \max_c p(z_i = c | x_i)$. But the GMM parameters are unknown.

Maximum Likelihood Estimation (MLE) is a method of estimating the parameters achieved by maximizing a likelihood function. But, for a general case, a closed form solution is not guaranteed for the parameters of GMM.

Expectation-Maximization Algorithm

The Expectation-Maximization (EM) algorithm is a simpler way to estimate the parameters of the GMM. The first step in the algorithm is to randomly initialize the parameters π_k , μ_k and Σ_k for $k = 1, \dots, K$.

- **Expectation Step (E-Step):** Compute the posterior probabilities that each data instance belongs to each Gaussian component:

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \mathcal{N}(x_i | \mu_j, \Sigma_j)}$$

This gives us a soft assignment of instances to components (called responsibilities).

- **Maximization Step (M-Step):** Update the parameters based on the responsibilities:

$$\begin{aligned}\pi_k &= \frac{N_k}{N} \\ \mu_k &= \frac{1}{N_k} \sum_{i=1}^N \gamma_{ik} x_i \\ \Sigma_k &= \frac{1}{N_k} \sum_{i=1}^N \gamma_{ik} (x_i - \mu_k)(x_i - \mu_k)^\top\end{aligned}$$

where $N_k = \sum_{i=1}^N \gamma_{ik}$ and N is the number of data instances.

The two steps are repeated until the model reaches a stable solution as the accuracy improves with each iteration. A stable solution is said to be reached when the parameters stop changing significantly or the log-likelihood stabilizes.

Advantages:

- **Flexibility:** Can model clusters with different shapes and sizes due to the covariance matrix.
- **Probabilistic Nature:** Provides a measure of uncertainty in cluster assignments.

Disadvantages:

- **Assumption of Gaussian Distribution:** Assumes that data follows a Gaussian distribution, which might not be true for all datasets.
- **Sensitivity to Initialization:** The results can be sensitive to the initial parameter values and may converge to local optima.

2.5 Evaluation Metrics for Clustering

Following is a discussion of appropriate methods to quantify the quality of clustering results. Based on the availability of ground truth labels, they can be classified into two kinds.

2.5.1 Internal Metrics

These metrics do not require ground truth labels. Instead, intrinsic properties of the clusters such as cohesion or separation are used to quantify the clustering quality.

Centroid Standard Deviation

Centroid Standard Deviation measures the variability of cluster centroids from their mean position across all clusters. It takes positive real number values. Low centroid standard deviation means the centroids are close together - possibly poor separation between clusters. High centroid standard deviation means the centroids are well-separated - suggests better-defined clusters.

It is a very simple measure that works well for spherical clusters of uniform density. But it is not scale-invariant and can be misleading if clusters are of different densities or non-spherical.

Silhouette Score

Silhouette Score measures how similar a instance is to its own cluster compared to other clusters.

For each instance i in the dataset,

- The average distance between i and all other instances in the same cluster as i is called the intra-cluster distance, $a(i)$.
- The average distance between i and all other instances in the nearest cluster to i , but not its own, is called the inter-cluster distance, $b(i)$.
- The silhouette score for i ,

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

Silhouette score takes values between $+1$ and -1 . A score of $+1$ means the instance is well-matched to its own cluster and poorly matched to the neighboring clusters, while a score of -1 means the instance is likely assigned to the wrong cluster. A score of 0 means the instance is on or very close to the decision boundary between two clusters.

It is independent of cluster centroid positions or shapes. It is a more useful and holistic measure of cluster quality, albeit more computationally expensive. But it does assume convex clusters.

2.5.2 External Metrics

These metrics evaluate clustering quality by comparing predicted groupings to known labels.

Purity

For a clustering solution, purity is calculated as:

$$\text{Purity} = \frac{1}{N} \sum_k \max_j |C_k \cap L_j|$$

where

- N : Total number of data instances
- C_k : Cluster k
- L_j : Ground truth class j
- $|C_k \cap L_j|$: Number of items in cluster k that belong to class j

Essentially, you find the most common true class label for each cluster, then sum over all clusters the number of instances from that most common class. You divide by the total number of data instances to get the purity score, which ranges from 0 to 1. 1 indicates perfect clustering where each cluster contains only one class. Lower scores indicate more mixing of classes in clusters. While purity is a simple and intuitive metric, it is also biased towards more clusters. If each data instance is its own cluster, the purity is 1. Hence, the metric might not be very informative depending on the nature of the dataset.

Rand Index and Adjusted Rand Index

Rand Index measures the percentage of decisions that are correct between two clusterings. A “decision” here is whether a pair of elements is in the same or different clusters.

$$\text{Rand Index, } R = \frac{a + b}{\binom{n}{2}}$$

where

- a is the number of pairs of instances that are either in the same cluster in both clusterings or in different clusters in both clusterings. In confusion matrix terms, this corresponds to true positives and true negatives.
- b is the number of pairs of instances that are in the same cluster in one clustering and in different clusters in the other. In confusion matrix terms, this corresponds to false positives and false negatives.
- n is the total number of instances, which mean the total number of pairs is $\binom{n}{2}$.

Rand Index takes values in the range of 0 to 1, a higher value indicating a better clustering. It can be an effective metric, but it does not correct for chance. This is where the Adjusted Rand Index comes in. It corrects the Rand Index for chance grouping. This is a more accurate evaluation metric for comparing clusterings, especially when the number of clusters is different or the classes are imbalanced.

$$\text{Adjusted Rand Index, } R' = \frac{R - \bar{R}}{\max(R) - \bar{R}}$$

where:

- $\bar{R}$ is the expected Rand Index (the expected value of the Rand Index for random clusterings).
- $\max(R)$ is the Rand Index for perfectly matching clusterings and always takes the value 1.

Adjusted Rand Index takes values in the range of -1 to 1 . An Index of 1 means perfect agreement between the ground truth and the clustering being evaluated. 0 means random labeling and a value less than 0 implies the clustering is worse than random.

2.6 Variational Autoencoder

On the basis of how they handle the data distribution, models can be classified into two types: discriminative and generative. The AE in discussion so far is a discriminative one.

A discriminative AE attempts to model the data given the observations in a dataset. Given a large enough dataset and a relatively simple system to model, discriminative models perform quite well by exploiting correlations in the data available.

But discriminative models, in some cases, create overly simplistic models of the underlying system, particularly when given insufficient data. This could lead to overfitting and poor generalization.

This is where generative modeling shines. Generative models tackle the broader problem of learning a joint distribution over all the variables. A generative model simulates how the data is generated. The generative process naturally expresses causal relations of the system. This allows the model to generalize much better to new situations than mere correlations. These models are usually highly intuitive and interpretable.

The GMM clustering algorithm, discussed in subsection 2.4.4, is in fact a generative model. It defines a joint probability distribution over data by modeling it as a mixture of Gaussian components. New data instances can be sampled by first selecting a component and then generating from its Gaussian distribution. Such generated instances would be representative of the underlying structure of that data captured by the model.

The AE model discussed so far is a discriminative model because it learns to map inputs directly to outputs, typically reconstructing the same input, without modeling the underlying data distribution. Variational autoencoders (VAEs), on the other hand, are a class of generative models with latent variables. They explicitly learn a probabilistic latent space that can be sampled to generate new data similar to the training set.

Given a dataset of observed variables x , and a model with hidden latent variables z , the model describes a joint probability distribution over the observed and hidden variables $p_{\theta}(x, z)$ where θ represents the parameters by which the relationship between the observed and latent variables is characterized.

The objective is to compute the posterior distribution $p_{\theta}(z|x)$. The posterior distribution represents the updated belief about the latent variable's distribution after observing

the data. Using Bayes' theorem, the posterior probability is computed as:

$$p_{\theta}(z|x) = \frac{p_{\theta}(x, z)}{p_{\theta}(x)} = \frac{p_{\theta}(x|z)p_{\theta}(z)}{p_{\theta}(x)}$$

In this equation, $p_{\theta}(x|z)$ is called the likelihood function and $p_{\theta}(z)$ is called the prior.

$p_{\theta}(x)$ is called the marginal distribution arrived at by marginalizing over z , that is,

$$p_{\theta}(x) = \int p_{\theta}(x, z) dz$$

This integral, also called the evidence, does not necessarily have a closed form solution and is often intractable. For this reason, rather than compute it directly, the posterior distribution can be approximated using variational inference.

2.6.1 Variational Inference

Variational inference approaches the problem using optimization. A family of probability distributions is defined over the latent variables, controlled by adjustable “variational parameters.” The goal of the optimization is to identify the distribution within this family that is closest — in terms of KL divergence — to the true conditional distribution. This optimized variational distribution then acts as an approximation to the exact conditional distribution.

In the case of the intractable posterior $p_{\theta}(z|x)$, it is approximated using a variational model with parameters ϕ :

$$p_{\theta}(z|x) \approx q_{\phi}(z|x)$$

The log-likelihood of x , under the parameters θ , that is $\log p_{\theta}(x)$ is independent of the latent variable z . Hence, its expectation with respect to $q_{\phi}(z|x)$ equals the quantity itself. Using this property,

$$\log p_{\theta}(x) = E_{q_{\phi}(z|x)} [\log p_{\theta}(x)]$$

This can be expanded as:

$$\begin{aligned}
\log p_{\theta}(x) &= E_{q_{\phi}(z|x)} \left[\log \frac{p_{\theta}(x, z)}{p_{\theta}(z|x)} \right] \\
&= E_{q_{\phi}(z|x)} \left[\log \frac{p_{\theta}(x, z)}{q_{\phi}(z|x)} \frac{q_{\phi}(z|x)}{p_{\theta}(z|x)} \right] \\
&= \underbrace{E_{q_{\phi}(z|x)} \left[\log \frac{p_{\theta}(x, z)}{q_{\phi}(z|x)} \right]}_{\text{ELBO}} + \underbrace{E_{q_{\phi}(z|x)} \left[\log \frac{q_{\phi}(z|x)}{p_{\theta}(z|x)} \right]}_{\text{KL Div}}
\end{aligned}$$

The first term in the equation is the Evidence Lower Bound(ELBO) term and the second term is Kullback-Leibler divergence(KL divergence) term. It is instructive to look at these terms in detail and understand what they do.

2.6.2 Kullback-Leibler Divergence

KL divergence is a statistical measure used to compare probability distributions. Given a true probability distribution P defined over a random variable x and a model probability distribution Q defined over the same variable, KL divergence measures how different P is from Q . It is defined mathematically as:

$$D_{\text{KL}}(P||Q) = \begin{cases} \sum_{x \in X} p(x) \log \frac{p(x)}{q(x)} & \text{for discrete distributions,} \\ \int_{-\infty}^{\infty} p(x) \log \frac{p(x)}{q(x)} dx & \text{for continuous distributions} \end{cases}$$

In probability theory, KL divergence is considered a statistical distance. A statistical distance quantifies the distance between two statistical objects, such as random variables, samples and distributions. But it is not considered a metric. Of the four conditions required for a distance to be a metric, KL divergence satisfies two. It is non-negative. And it satisfies the identity of indiscernibles. That is, $D_{\text{KL}}(P||Q) = 0$ if and only if $P = Q$. But it fails to satisfy the condition of symmetry, that is, $D_{\text{KL}}(P||Q) \neq 0$ for all P and Q . It also does not satisfy the triangle inequality condition, which means for P , Q and R , $D_{\text{KL}}(P||R) \geq D_{\text{KL}}(P||Q) + D_{\text{KL}}(Q||R)$

2.6.3 ELBO Function

The ELBO function, also known as the variational lower bound, is the optimization objective of the VAE. The ELBO can be rewritten to reveal its two key components, offering intuition about its optimization:

$$\begin{aligned}\mathcal{L}_{\theta,\phi}(x) &= E_{q_\phi(z|x)}[\log p_\theta(x, z)] - E_{q_\phi(z|x)}[\log q_\phi(z|x)] \\ &= E_{q_\phi(z|x)}[\log p_\theta(x|z)] + E_{q_\phi(z|x)}[\log p_\theta(z)] - E_{q_\phi(z|x)}[\log q_\phi(z|x)] \\ &= E_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}(q_\phi(z|x) || p_\theta(z))\end{aligned}$$

- Expected Log-Likelihood ($E_{q_\phi(z|x)}[\log p_\theta(x|z)]$): This term, often called the "reconstruction error" or "negative reconstruction error" in the VAE context, encourages the generative model $p_\theta(x|z)$ to assign high probability to the observed data x for the sampled latent variables z . It biases the model towards configurations of latent variables that explain the observed data.
- Negative KL Divergence between approximate posterior and prior ($-D_{KL}(q_\phi(z|x) || p_\theta(z))$): This term acts as a regularizer. It encourages the approximate posterior $q_\phi(z|x)$ to stay close to the prior distribution $p_\theta(z)$ over the latent variables. Without this term, the optimization objective depends solely on reconstruction accuracy. In this setting, stochasticity in the latent variable degrades reconstruction performance, so the optimal solution is to reduce the posterior variance toward zero, causing $q_\phi(z|x)$ to collapse to a deterministic point estimate. The KL term counteracts this tendency by enforcing proximity to the prior, thereby preserving uncertainty and preventing degeneration into a standard autoencoder.

Lower Bound on Marginal Likelihood

Since $D_{KL}(q_\phi(z|x) || p_\theta(z|x)) \geq 0$, it directly follows that $\mathbf{L}_{\theta,\phi}(\mathbf{x}) \leq \log \mathbf{p}_\theta(\mathbf{x})$. This is why it's called the "lower bound". The KL divergence term also represents the "gap" between

the ELBO and the true log marginal likelihood; a smaller KL divergence implies a tighter bound.

Dual Optimization Goal

Maximizing the ELBO with respect to both generative model parameters (θ) and inference model parameters (ϕ) concurrently achieves two important goals:

- It approximately maximizes the marginal likelihood $p_\theta(x)$, thereby improving the generative model.
- It minimizes the KL divergence $D_{KL}(q_\phi(z|x)||p_\theta(z|x))$, making $q_\phi(z|x)$ a better approximation of the true posterior $p_\theta(z|x)$.

Relation to EM Algorithm

The ELBO is closely related to the EM algorithm discussed in the GMM section. The ELBO's first term (expected complete log-likelihood, $E_{q(z|x)}[\log p(z, x)]$) is similar to what EM optimizes. Variational inference applies to models where the exact conditional of latent variables cannot be computed. In this sense, the EM algorithm can be thought of as a special case of ELBO optimization where $q(z)$ is set to the exact posterior.

2.6.4 ELBO optimization and the Reparametrization trick

The ELBO is particularly useful because it allows for joint optimization with respect to all the parameters, both θ and ϕ .

To calculate the gradient of the ELBO with respect to θ :

$$\begin{aligned}\nabla_{\theta} \mathcal{L}_{\theta, \phi}(x) &= \nabla_{\theta} E_{q_{\phi}(z|x)}[\log p_{\theta}(x, z)] - \nabla_{\theta} E_{q_{\phi}(z|x)}[\log q_{\phi}(z|x)] \\ &= E_{q_{\phi}(z|x)}[\nabla_{\theta} \log p_{\theta}(x, z)] - 0\end{aligned}$$

Calculating the gradient of the ELBO with respect to ϕ , on the other hand, is not straightforward. In this case, the gradient of the expectation is not equal to the expectation of the gradient because the ELBO's expectation is taken with respect to the distribution $q_\phi(z|x)$, which is a function of ϕ .

In order to address this issue, VAEs use the reparameterization trick to obtain unbiased estimates of the ELBO's gradients.

Reparameterization trick

Given the random variable z and the parameters ϕ such that $z \sim q_\phi(z|x)$, z is expressed as :

$$z = g(\varepsilon, \phi, x)$$

where ε is a random variable whose distribution, $\varepsilon \sim p(\varepsilon)$, is independent of x or ϕ .

The trick is essentially a change of variables under which the expectations are rewritten in terms of ε ,

$$\begin{aligned}\mathcal{L}_{\theta, \phi}(x) &= E_{q_\phi(z|x)}[\log p_\theta(x, z)] - E_{q_\phi(z|x)}[\log q_\phi(z|x)] \\ &= E_{p(\varepsilon)}[\log p_\theta(x, z) - \log q_\phi(z|x)]\end{aligned}$$

2.6.5 Posterior Collapse

The posterior in a VAE serves two competing goals. It needs to be **informative**. That is, $q_\phi(z|x)$ should encode meaningful information about x , so that different inputs map to different regions of latent space. And it needs to be regularized: $q_\phi(z|x)$ should remain close to the prior $p(z)$, enabling smooth sampling and generalization. At optimum, the encoder should approximate the true posterior and maintain non-zero mutual information. This ensures the latent variable captures structure in the data while remaining regularized.

A Posterior is said to have collapsed when this balance fails and the optimal solution

under the ELBO causes the variational posterior to ignore the input, $q(z|x) \sim p(z)$. In this case, the KL divergence term approaches zero, and the latent variable becomes statistically independent of x . Consequently, the decoder learns to model the marginal distribution $p(x)$ directly, without conditioning meaningfully on z . Reconstruction is achieved almost entirely through the decoder's capacity, rather than through information passed via the latent space.

Some strategies to avoid posterior collapse will be discussed in the following chapter.

Chapter 3

Methodology

3.1 Tools used

Micromagnetic simulations being analyzed were generated using the `ubermag`[4] and `neuralmag`[7] framework. Built on top of simulation packages such as `OOMMF`[10] and `mumax3`[32], `ubermag` provides a Python interface for defining geometries, material parameters, and solving the Landau–Lifshitz–Gilbert equation. The machine learning pipeline was implemented in Python using `PyTorch`[25] to construct and train the AE models, with `Lightning`[11] employed to structure the training workflow, handle logging, and improve reproducibility. Unsupervised clustering in the learned latent space was carried out using algorithms from the `scikit-learn`[26] library, including Gaussian Mixture Models and DBSCAN. Hyperparameter optimization was partly done using Optuna. For analysis and visualization, `matplotlib`[18] was used for static figures, while `plotly`[19] was employed for the interactive exploration of latent spaces and clustered micromagnetic states.

3.2 Data

The magnetization configurations $m(r)$ are defined as vector fields mapping points from a spatial subregion $S \subset \mathbb{R}^n$ to a vector space $V \subset \mathbb{R}^n$ ($m : S \rightarrow V$), where S represents the

region occupied by the magnetic material, typically given in discretized form for numerical simulations. The field values are constrained by the material’s saturation magnetization. In the micromagnetic framework adopted here, the magnetization is normalized such that $|m(r)| = 1$ at every spatial point.

To assess the robustness and generality of the proposed unsupervised learning framework, three distinct micromagnetic datasets were analyzed. Each dataset corresponds to a different physical system, allowing the clustering methodology to be tested across varying geometries, material parameters, and variables. The vector fields in the datasets represent equilibrium magnetization configurations. It should be noted that the equilibrium state obtained is highly dependent on the initial state at the start of the simulation. Hence, for a given set of material parameters and variables, the simulations were run several times with multiple initial states.

3.2.1 Dataset 1

The first dataset consists of micromagnetic simulations of an iron germanium (FeGe) nanodisc with a diameter of 160 nm and a thickness of 10 nm. The disc is subjected to an external magnetic field varying from 0 to $3.18 \times 10^9 \text{ Am}^{-1}$, taking 345 randomly chosen values along the positive z -axis. The material parameters for the simulations are $A = 8.78 \times 10^{-12} \text{ J m}^{-1}$, $D = 1.58 \times 10^{-3} \text{ J m}^{-2}$, and $M_s = 384 \text{ kA m}^{-1}$.

The system was discretized using a cubic mesh with a cell size of 2 nm, resulting in an array of size $80 \times 80 \times 5 \times 3$, with three magnetization components stored at each spatial location. Because the disc thickness is small relative to the characteristic length scales over which magnetization variations occur in this material, the magnetization can be considered effectively uniform along the out-of-plane direction. Consequently, a single central slice provides a representative description of the full three-dimensional configuration and reduces the dimensionality to $80 \times 80 \times 3$. This procedure yielded a total of 3010 equilibrium magnetization configurations.

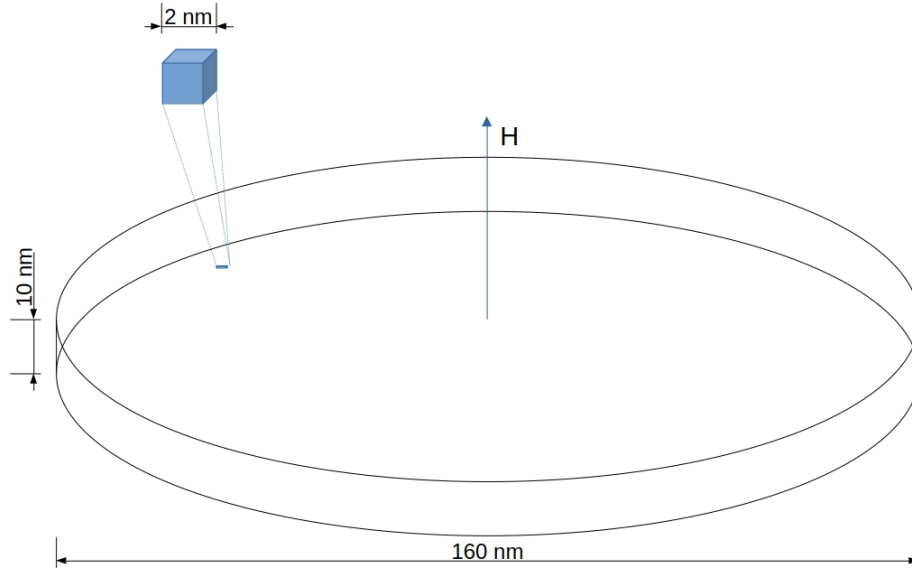


Figure 3.1: Geometry and discretization of Dataset1

3.2.2 Dataset 2

The second dataset consists of micromagnetic configurations generated from simulations of FeGe, similar to the previous dataset. Unlike the previous dataset, the geometry is a cube of edge 300 nm in length. The simulations are performed across a range of external magnetic field and temperature values. The temperature values range from 0 to 3000 K, and the magnetic field strength values range from 0 to $0.39 \times 10^6 \text{ Am}^{-1}$.

The raw simulation grids have dimensions $97 \times 97 \times 97$, with a cubic mesh of size 3.125 nm. With three magnetization components per cell, Each magnetic state has a $97 \times 97 \times 97 \times 3$ dimensional representation. The dataset has a size of 2601.

Preprocessing and Data Augmentation

Average pooling computes the average of the elements present in the region of the feature map covered by the filter. For this use case, average pooling solves two issues. It reduces the dimensionality, making training less computationally intensive. It also smooths the small-scale noise induced by the finite temperature of the system while preserving large-scale magnetic structures. Hence, preprocessing is applied in the form of three-

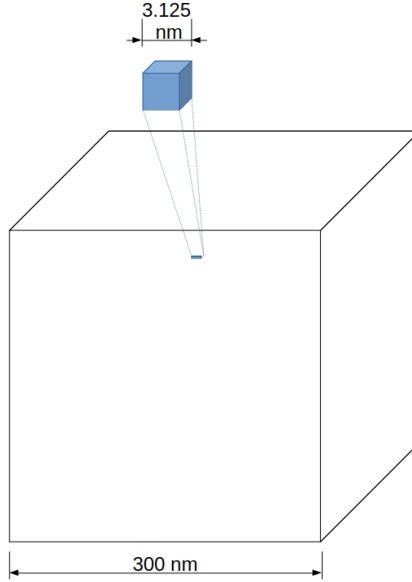


Figure 3.2: Geometry and discretization of Dataset2

dimensional adaptive downsampling using average pooling with kernel size 7, stride 2, and padding 2, reducing the resolution to $48 \times 48 \times 48 \times 3$.

To improve invariance with respect to in-plane rotations and increase the effective dataset size, data augmentation is performed by randomly rotating each configuration by multiples of $\pi/2$ about the z-axis. Hence, each physical sample contributes four rotated versions.

3.3 Autoencoder Implementation

The AE network consists of an encoder, which maps high-dimensional magnetization fields into a low-dimensional latent representation, and a decoder, which reconstructs the original field from the latent space. The decoder architecture mirrors the encoder in reverse. The overall training objective is to minimize the mean squared error (MSE) between the input and reconstructed configurations, thereby preserving the essential features of the

magnetization while reducing dimensionality. The encoder and decoder models are structured as a set of blocks applied in series.

3.3.1 Convolutional Block

Convolutional blocks constitute the core feature extraction layers of the encoder. Each block consists of:

1. Convolutional Layer (`torch.nn.Conv2d`) – Applies multiple learnable filters to the input tensor to extract spatial features from the magnetization field, capturing local patterns such as vortices or skyrmion cores. Stride and kernel size control the receptive field and downsampling.
2. Transposed Convolution (`torch.nn.ConvTranspose2d`) – Increases the spatial dimensions of feature maps while learning reconstruction filters. This reverses the downsampling performed in the encoder. This layer takes the place of the Convolutional layer in the decoder model.
3. Batch Normalization (`torch.nn.BatchNorm2d`) – Normalizes the output of the convolution to stabilize and accelerate training, mitigating internal covariate shift[20].
4. Activation Function (GELU) – Applies the Gaussian Error Linear Units function. $\text{GELU}(x) = x\Phi(x)$ where $\Phi(x)$ is the cumulative distribution function for the standard normal distribution. GELU improves neural network performance by enabling smoother gradient flow, mitigating the vanishing gradient problem, and capturing complex patterns through its non-monotonic, probabilistic activation mechanism[14]. It is preferred in deep architectures and computer vision tasks.

3.3.2 Linear Block

Linear (fully connected) blocks operate after the convolutional layers to compress or expand the flattened feature vectors between the encoder, latent space, and decoder. Each linear block contains:

1. Linear Layer (`nn.Linear`) – Performs an affine transformation, mapping the input vector to a specified output dimension. In the encoder, this reduces dimensionality toward the latent space; in the decoder, it expands the latent representation back to the original feature size.
2. Activation Function (LeakyReLU) – Introduces nonlinearity to enable the network to model complex interactions between features. It is a popular choice due to its computational efficiency. It introduces a slight modification to the standard ReLU by assigning a small, fixed slope to the negative part of the input. This ensures that neurons do not become inactive during training, as they can still pass small gradients even when receiving negative values. $\text{Leaky ReLU}(x) = \begin{cases} x & \text{if } x > 0, \\ 0.01x, & \text{if } x \leq 0 \end{cases}$

3.3.3 Flatten and Global Pooling

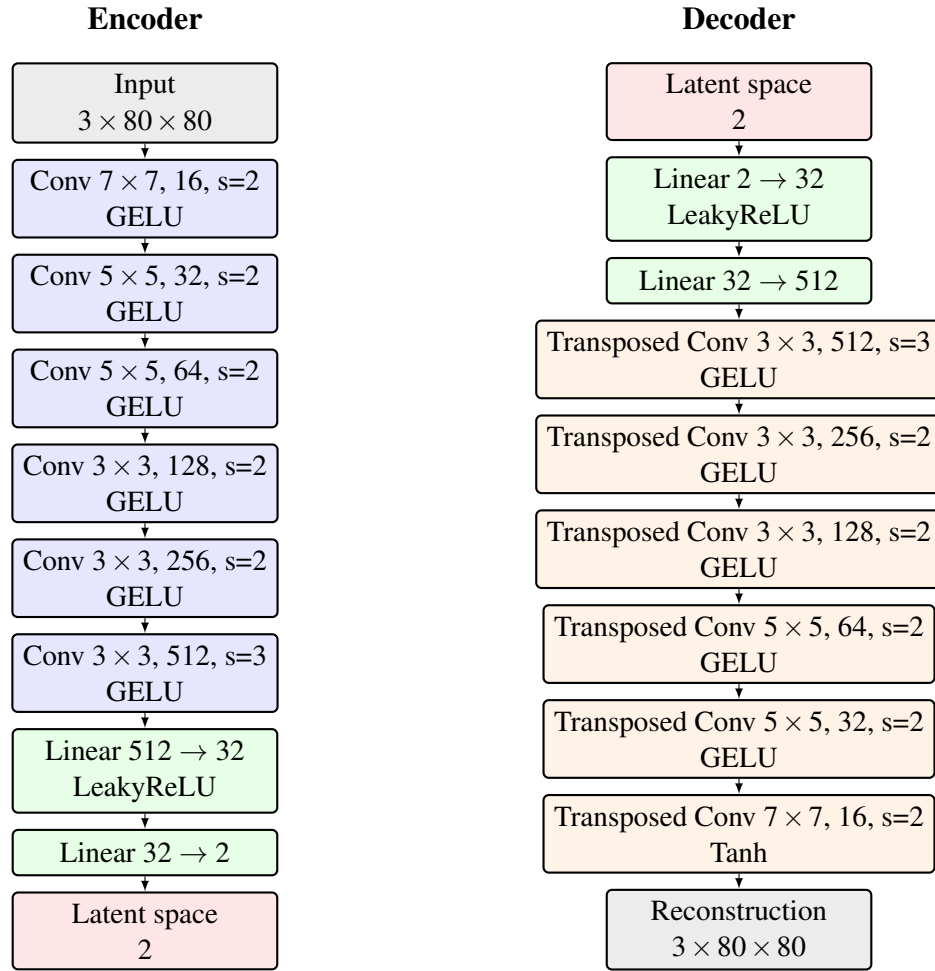
To interface between convolutional and linear layers, the network uses Flatten layers in the encoder to convert multi-channel feature maps into one-dimensional vectors. In the decoder, Unflatten layers restore the spatial structure.

Additionally, Global Average Pooling (GAP) is applied to some of the models, followed by nearest-neighbor upsampling in the decoder to recover the original dimensions. GAP computes the average of all spatial values in each feature map, summing the entire map into one scalar per channel. GAP retains positional information through channel ordering[21].

3.3.4 Final Tanh layer

The final output layer of the model, which is the activation function of the last convolutional block in the decoder, is a Tanh layer. This function compresses large positive values close to 1 and large negative values close to -1 . This range is physically meaningful for normalized magnetization values.

3.4 Network architecture for Dataset 1

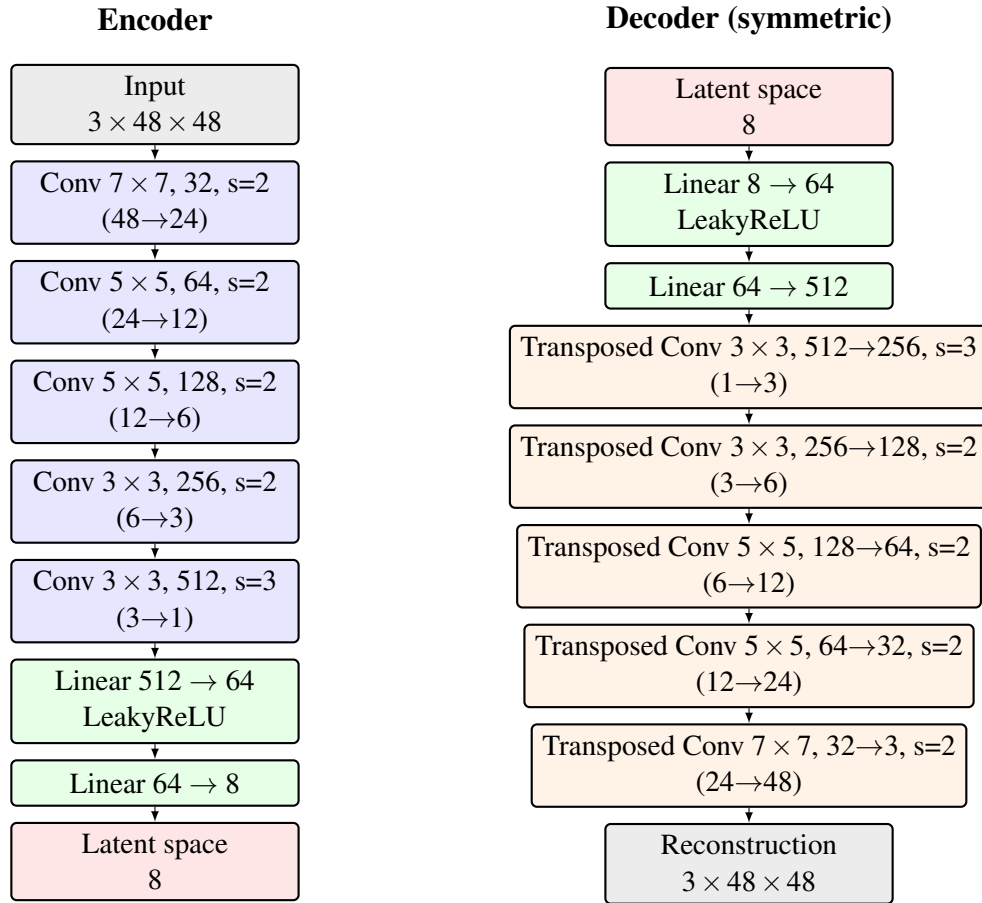


The AE and VAE share the same architecture apart from the layers relevant to the reparametrization trick discussed in the next section.

Hyperparameter	Value
Learning rate	0.001
Batch size	64
Epochs	200
Threshold for VAE	4

Table 3.1: Training hyperparameters

3.5 Network architecture for Dataset 2



The AE and VAE share the same architecture apart from the layers relevant to the reparametrization trick discussed in the next section.

Hyperparameter	AE Value	VAE value
Learning rate	0.001	0.001
Batch size	512	512
Epochs	1000	2000
Threshold for VAE	-	16

Table 3.2: Training hyperparameters

3.6 Variational Autoencoder

The VAE model inherits the architecture from the previously described convolutional AE and is extended to capture a probabilistic latent representation of the magnetization configurations.

3.6.1 Latent Space Parameterization

Instead of mapping inputs directly to a single latent vector, the VAE encodes each configuration into a probabilistic representation defined by a mean vector μ and a log-variance vector $\log \sigma^2$. These parameters are produced by two separate fully connected layers applied to the encoder output. To promote stable training, the weights of both fully connected layers are explicitly initialized using Xavier (Glorot) uniform initialization. The bias of the mean layer is set to zero, which initially centers the latent distribution. The bias of the log-variance layer is initialized to a negative value, corresponding to a small initial variance. This encourages the encoder to begin with compact latent representations and prevents excessive stochasticity in the early stages of training, improving the convergence and stability of the VAE.

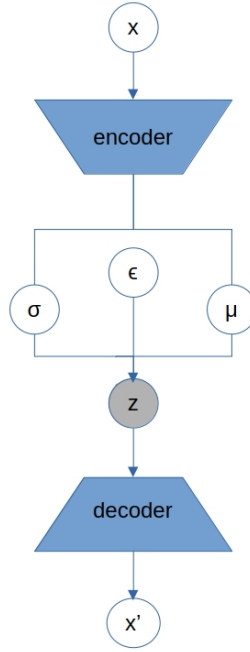


Figure 3.3: Reparametrization trick

3.6.2 Reparameterization Trick

To enable backpropagation through the stochastic sampling process, the VAE employs the reparameterization trick. A latent vector z is sampled such that

$$z = \mu + \epsilon \odot \sigma \quad (3.1)$$

where $\odot$ is the element-wise product, $\epsilon \sim \mathcal{N}(0, I)$ and $\sigma = \exp(0.5 \log \sigma^2)$. This separates the stochastic component from the network parameters, allowing gradients to propagate through μ and $\log \sigma^2$ during training while maintaining a probabilistic latent representation.

3.6.3 Loss Function and Regularization

Training the VAE involves minimizing a composite objective function composed of a reconstruction loss and a regularization term. The reconstruction loss is defined as the mean squared error between the input magnetization field and its reconstruction, encouraging accurate reproduction of the original configurations. In addition, the Kullback–Leibler divergence between the learned latent distribution and a standard normal prior is minimized. This regularization constrains the latent space to remain smooth and approximately Gaussian, preventing overfitting and ensuring that the learned representation is suitable for clustering.

3.6.4 Latent Representation Extraction

After training, the encoder is used to project each magnetization configuration into latent space. Rather than sampling, the mean vector μ is taken as the deterministic embedding for each input. This ensures reproducibility and reduces stochastic noise in the clustering stage.

3.7 Mitigating Posterior Collapse

The posterior collapse problem discussed in 2.6.5 can be tackled in number of ways. Following is a discussion of three such mitigation strategies :

1. **KL-weighting** : The ELBO function in a VAE balances reconstruction accuracy against regularization of the approximate posterior toward the prior. KL-weighting shifts this balance by introducing a scalar coefficient β on the KL term,

$$\mathcal{L}_\beta = E_{q_\phi(z|x)}[\log p_\theta(x|z)] - \beta D_{KL}(q_\phi(z|x) || p_\theta(z))$$

By tuning β , the relative importance of reconstruction and regularization can be controlled. Values $\beta < 1$ relax the pressure to match the prior, encouraging the

model to encode more information in the latent variables.

KL-weighting is a simple yet effective mechanism to influence latent utilization, but a fixed β does not adapt over the course of training and therefore may still permit posterior collapse when powerful decoders are used.

2. **KL-annealing** : This approach is similar to the previous one, but improves on it by making the β term time-dependent. Instead of applying the full KL penalty from the beginning of training, $\beta(t)$ is gradually increased from a small value (often near zero) to its final target value. Early in training, the model focuses primarily on reconstruction, allowing the encoder to learn informative latent representations without being immediately forced to match the prior. As training progresses, the KL term is progressively enforced, encouraging smoothness and regularization of the latent space.
3. **Free bits / KL thresholding** : This is a strategy in which the Kullback–Leibler (KL) divergence term is constrained by a minimum target value, rather than being uniformly weighted throughout training. Concretely, the KL contribution is suppressed when it falls below a predefined threshold and enforced only once it exceeds that level. This prevents latent dimensions from collapsing prematurely to the prior, while still allowing meaningful regularization once sufficient information is encoded.

The results presented in the next section use this strategy. A combination of KL-weighting and KL-annealing was also tried. Unlike that approach, which gradually increases the KL weight over training and can be sensitive to schedule design and timing, KL-thresholding directly enforces a minimum information capacity, making it less dependent on heuristic scheduling.

3.8 Training Procedure

Dataset 1 was split into training, validation, and test sets during training. 20% of the dataset was used for validation and 12% for testing. The validation set was useful for

selecting the architecture, tuning hyperparameters, and monitoring for overfitting. The test set was used for reporting the MSE loss metric on unseen data after training.

A test set was not used for datasets 2 and 3. Since the task at hand is clustering, the crucial metric is clustering quality and not MSE loss. Also, Generalization to unseen data is not a priority.

Training was performed using the Adam optimizer, which provides adaptive learning rates for efficient convergence in high-dimensional parameter spaces[23].

To ensure the selection of a well-converged model while avoiding overfitting, Lightning’s checkpointing mechanism was used in place of early stopping. During training, model checkpoints were automatically saved to capture the network parameters corresponding to the lowest validation loss. After training, the checkpoint with the best recorded validation performance was identified and loaded for subsequent latent space encoding. This approach guaranties that the model used for clustering represents the most accurate reconstruction observed during training, while avoiding potential issues where early stopping might terminate training prematurely.

3.9 Hyperparameter Optimization

Hyperparameter optimization with Optuna is performed through an adaptive, trial-based search strategy that efficiently explores the hyperparameter space to identify configurations that optimize a predefined objective function. An objective function is defined that trains and evaluates a model, while Optuna dynamically samples hyperparameters from specified distributions during each trial. Unlike grid or random search, Optuna employs Bayesian optimization via the Tree-structured Parzen Estimator (TPE), which models the relationship between hyperparameter values and objective performance to guide subsequent sampling toward more promising regions of the search space. Over multiple trials, the study accumulates performance observations and progressively refines its sampling strategy, ultimately returning the hyperparameter configuration that achieves the best objective value according to the defined metric.

Optuna was useful in refining the hyperparameters of the simpler Autoencoder models.

But, it was most effective in exploring a large search space to find the optimal parameters for the clustering algorithms using the clustering evaluation metrics.

Chapter 4

Results and Outlook

4.1 Dataset 1

4.1.1 Model Reconstruction Performance

To ascertain that the latent space obtained from the model is informative, the most demonstrative metric is the reconstruction quality. Quantitatively, reconstruction quality can be evaluated by plotting the training and validation losses of the model against the number of epochs for which the model is trained.

Figure 4.1 shows the evolution of the training and validation loss of the autoencoder as a function of the training epoch. Both losses decrease rapidly during the initial phase of training, dropping by more than an order of magnitude within the first 10–15 epochs, indicating fast learning of the dominant structures. After this transient regime, the losses continue to decrease more gradually and stabilize at low values, with only minor fluctuations. The close agreement between the training and validation curves throughout training suggests that the model generalizes well and does not exhibit significant overfitting. After approximately 50 epochs, the improvement becomes marginal, indicating convergence of the network and a stable latent representation.

Figure 4.2 presents the evolution of the total training and validation loss of the VAE,

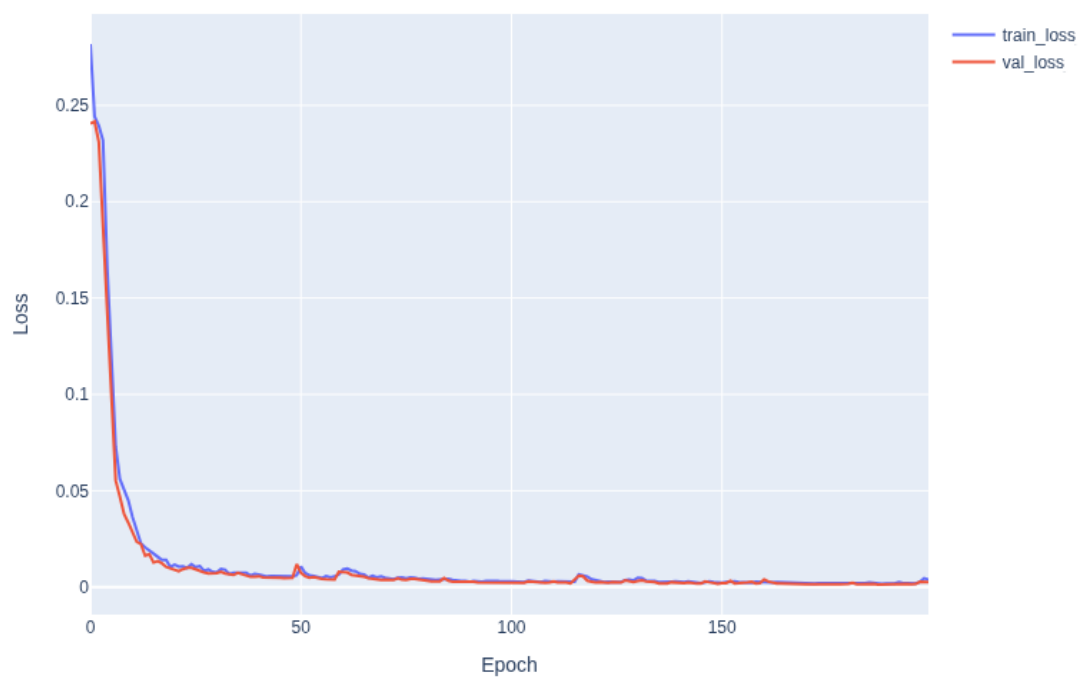


Figure 4.1: AE Loss vs Epoch, Dataset 1

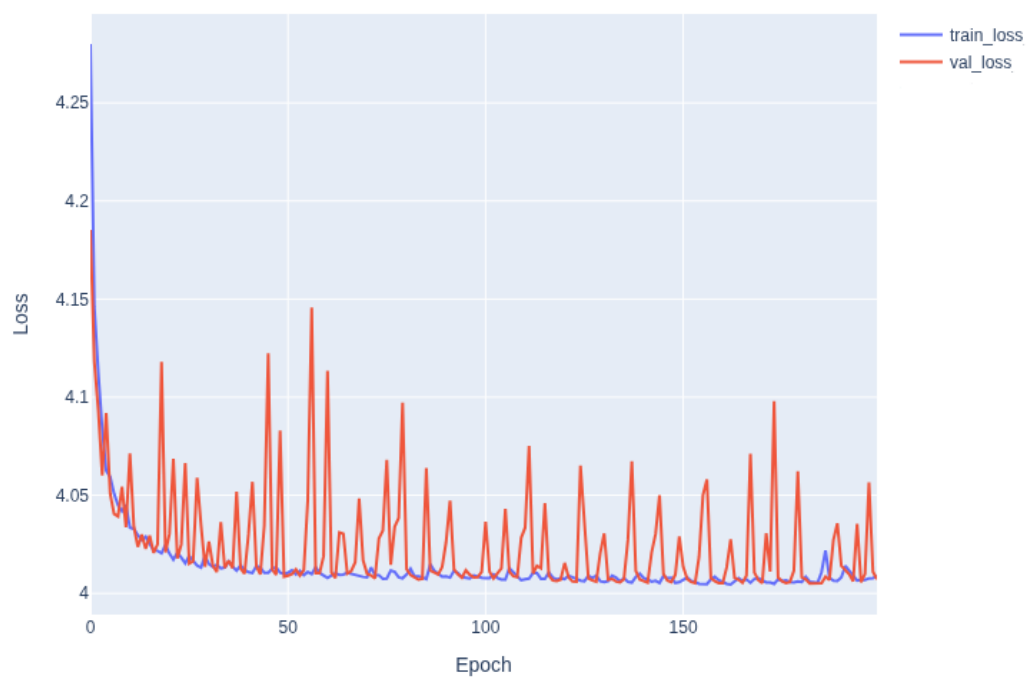


Figure 4.2: VAE Loss vs Epoch, Dataset 1

which are the sums of its reconstruction and Kullback–Leibler (KL) divergence components. As in the previous case, the total loss decreases sharply during the first epochs, reaches a plateau, and remains stable over the remainder of training. The training and validation curves remain closely aligned, demonstrating consistent generalization behavior. Noticeable oscillations are present in the validation loss, which arise from the use of KL-thresholding during training. As discussed in section 3.7, this mechanism enforces a minimum KL contribution, leading to intermittent activation of the regularization term and resulting in step-like fluctuations in the validation curve.

During training, each batch loss is evaluated under a slightly different set of weights, as optimization progresses within the epoch. When these batch losses are averaged, the result is a comparatively smooth training curve. In contrast, validation loss is computed after the epoch using a fixed set of parameters over a smaller validation set. Consequently, training loss is smoother than validation loss.

Looking at 4.2 and 4.3 in conjunction, It should be noted that the total Loss in the former graph is not the sum of the components in the latter graph. Due to the KL-thresholding mechanism,

$$\text{total loss} = \text{MSE loss} + \max(\text{threshold} + \text{KL divergence})$$

Also, the difference in scale makes the peaks in 4.2 appear more drastic.

It is also important to evaluate the reconstruction qualitatively by comparing the original magnetization fields with the reconstructed fields. The visualizations consist of two superimposed representations: a vector field and a scalar field. The vector plot displays the two in-plane components of the magnetization as arrows, representing the projection of the three-dimensional vector field onto the selected plane. Since this projection does not capture the out-of-plane contribution, the z -component is shown separately as a scalar field using the RdBu color map, with the corresponding color bar indicating its magnitude and sign.

A direct comparison between the standard AE and the VAE, as can be seen in 4.4, reveals no significant differences in reconstruction performance or convergence behavior. Both models achieved comparable loss values and demonstrated similar generalization

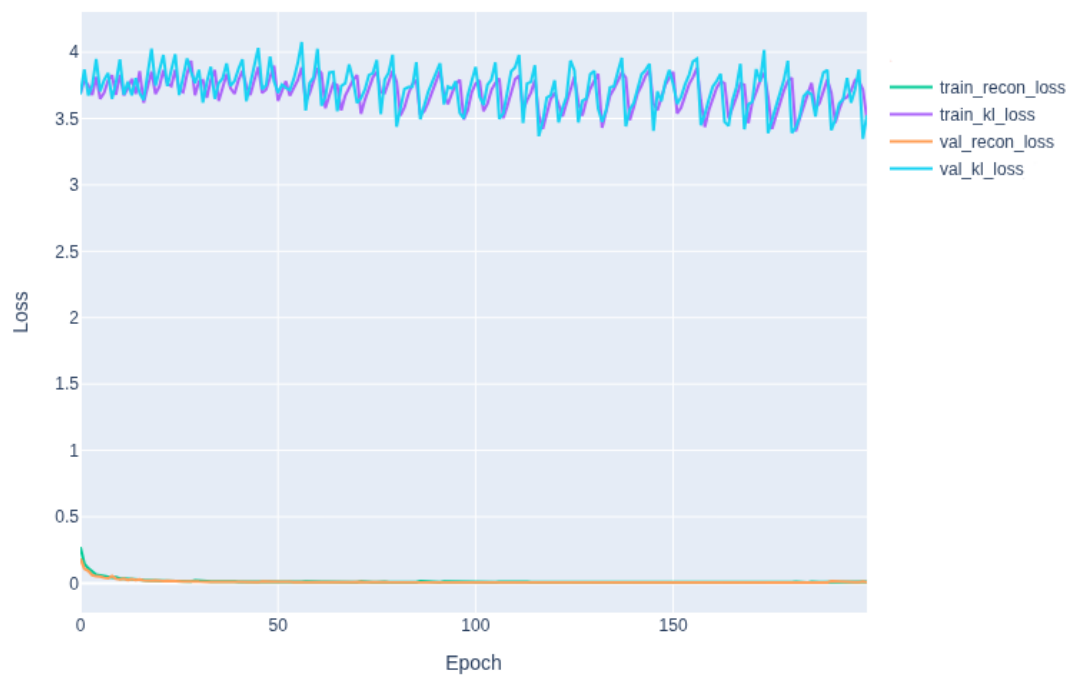


Figure 4.3: VAE MSE Loss and KL Divergence Loss vs Epoch, Dataset 1

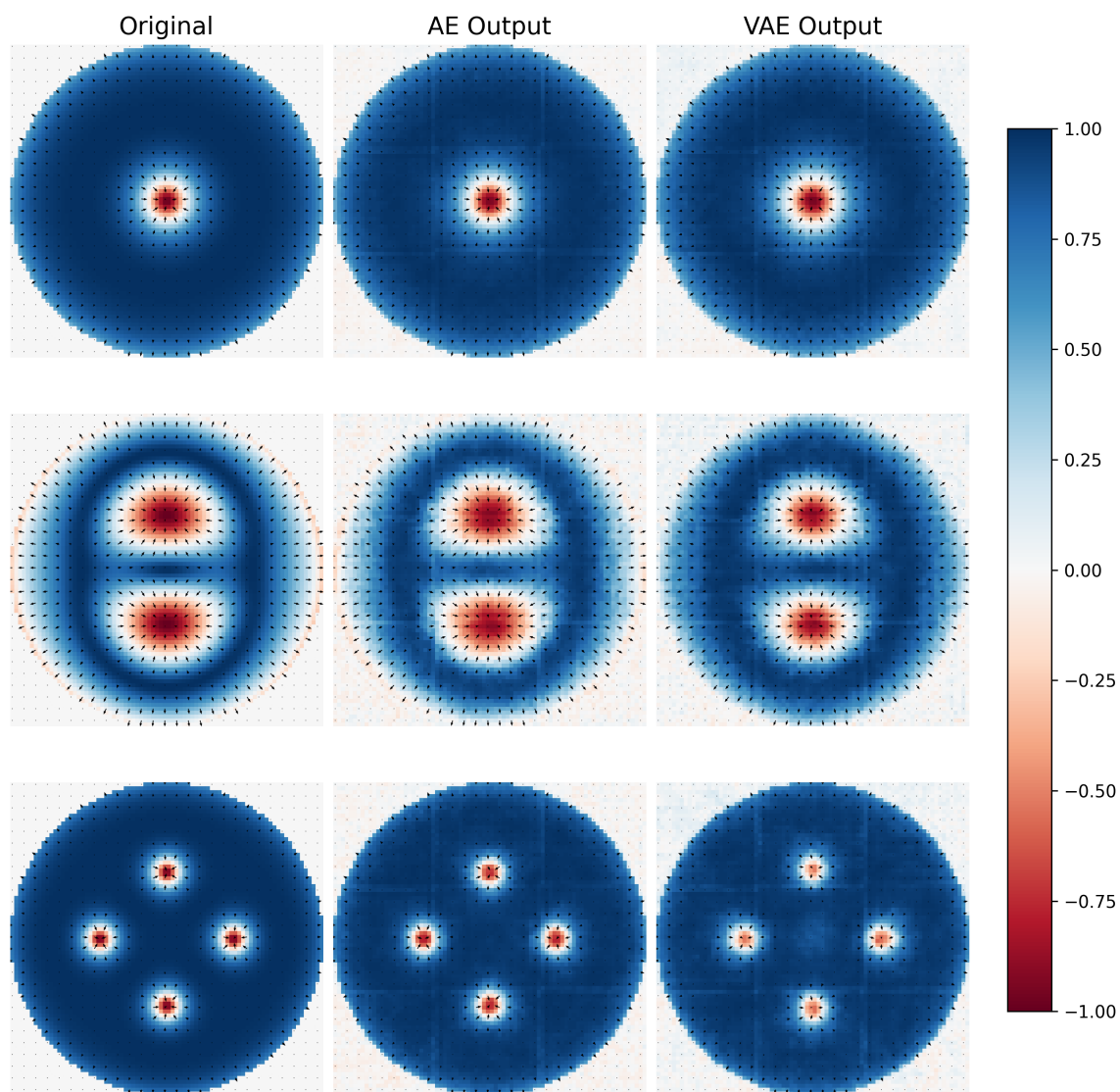


Figure 4.4: Comparison of original and reconstructed field images, Dataset 1

characteristics. For the sake of brevity, the subsequent analysis focuses on the latent space structure learned by the VAE.

4.1.2 Clustering Evaluation

To quantitatively assess the clustering quality of the VAE-based latent representations, the dataset was manually labeled according to distinct micromagnetic state categories. This labeling provides a reference against which the unsupervised clustering results can be evaluated. The resulting class distribution is highly imbalanced. A small number of dominant classes accounts for the majority of configurations, while several classes contain only a limited number of samples.

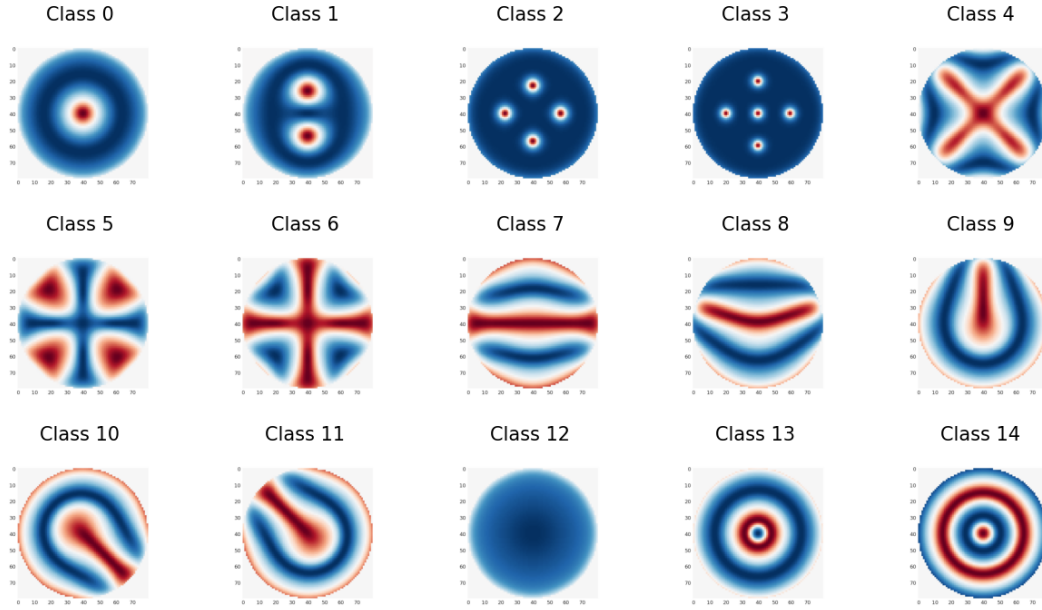


Figure 4.5: Caption

DBSCAN was applied to the latent representations using parameters $\varepsilon = 0.5$ and a minimum of 5 samples per cluster. The resulting clustering exhibits excellent agreement with the manually assigned labels, achieving a purity of 0.999 and an adjusted Rand index (ARI) of 0.9988. These values indicate an almost perfect correspondence between the un-

supervised clusters and the reference micromagnetic classes. The high ARI demonstrates that the density-based clustering effectively captures the underlying structure of the latent space while correctly identifying both dominant and less-populated magnetic states, as well as separating outlier configurations.

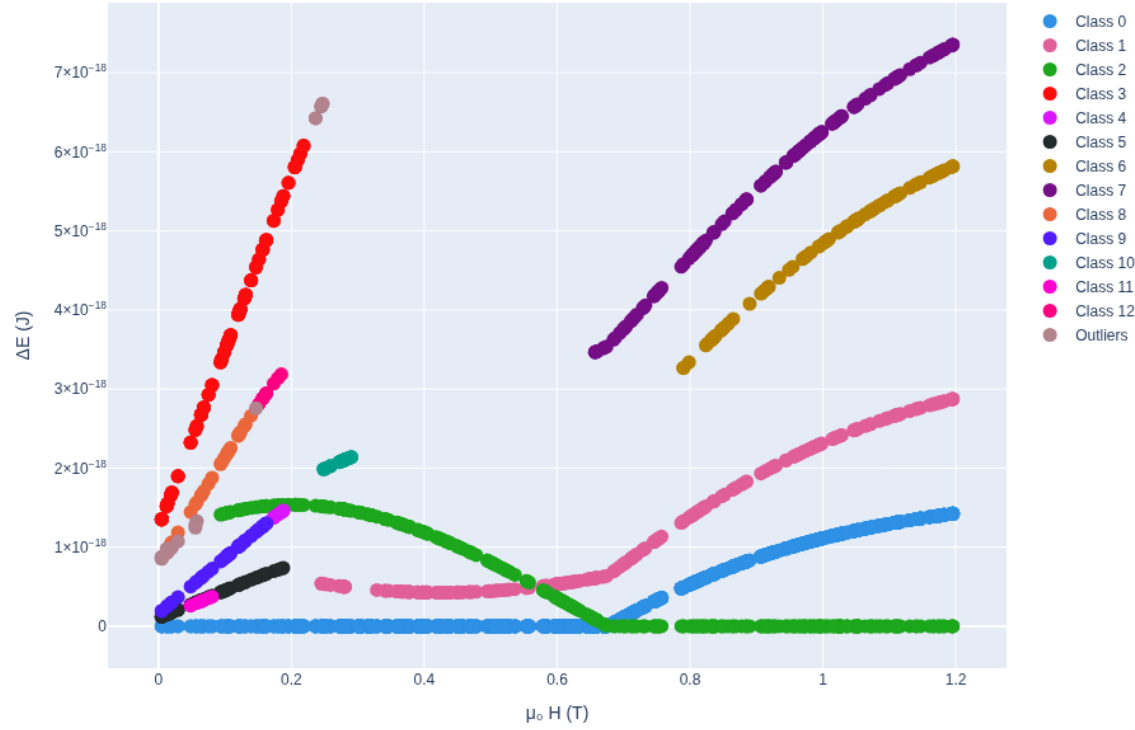


Figure 4.6: Clustering results plotted as a function of the applied external magnetic field on the vertical axis and the energy difference of each configuration relative to the ground state at the corresponding field value on the horizontal axis.

4.1.3 Latent Space Structure

Figure 4.7 shows the two-dimensional latent space learned by the VAE, with points colored according to the cluster labels assigned by DBSCAN. The clustering reveals 13 well-separated groups, each forming a compact and coherent structure in the latent representation. The clear separation between clusters indicates that the VAE successfully embeds

physically distinct magnetization configurations into distinct regions of latent space. In addition to the clustered points, DBSCAN identifies a small number of configurations as noise, corresponding to outlier states that do not belong to any dense region of the latent space. These outliers can be accounted for by the aforementioned sparse classes, which are underrepresented in the dataset. The arbitrary shapes of the clusters make DBSCAN a good choice.

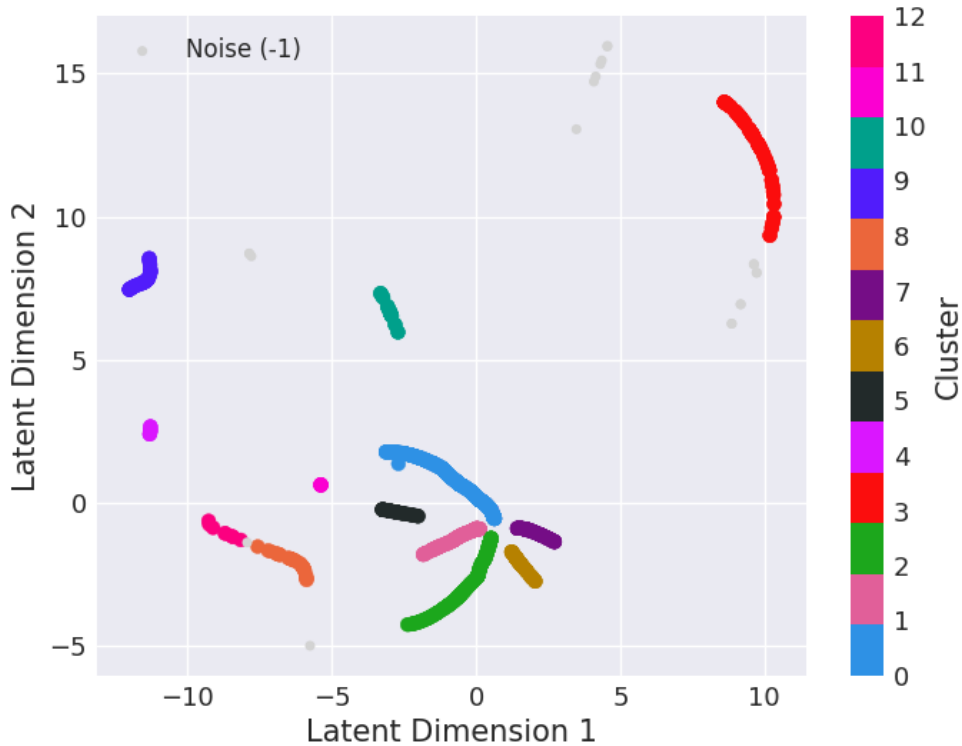


Figure 4.7: Vizualization of the latent space with DBSCAN-assigned classes

4.1.4 Generative Exploration of the Latent Space

Figure 4.8 shows the reconstructed magnetization configurations obtained by uniformly sampling from the learned latent space. Each point on the grid corresponds to a specific

pair of latent coordinates. The horizontal and vertical axes represent the first and second latent dimensions, respectively.

A clear and structured organization of magnetic patterns emerges across the latent plane. Smooth transitions between distinct micromagnetic configurations are observed when moving continuously through latent space, indicating that the VAE has learned a meaningful and continuous embedding of physically related states. Different configurations occupy different regions of the plane, demonstrating that the latent variables encode physically relevant features of the magnetization field. The continuity of transformations between neighboring samples further confirms that the latent representation captures systematic variations rather than isolated configurations.

4.2 Dataset 2

4.2.1 Model Reconstruction Performance

The results for Dataset 2 are presented following the same structure and evaluation procedure as described for Dataset 1.

The training and validation loss curves for Dataset 2 are shown in Figure 4.9. Both losses decrease rapidly during the initial training phase, indicating that the network quickly captures the dominant structural features of the magnetization configurations. Following this early reduction, the loss decreases more gradually over several hundred epochs, reflecting the refinement of the latent representation.

Around epoch 550–600, a pronounced drop in both training and validation loss is observed, after which the curves converge toward a stable plateau. This behavior suggests a transition to a more accurate reconstruction regime, potentially associated with improved latent space organization. The training and validation curves remain closely aligned throughout the process, with only minor fluctuations in the validation loss. This indicates stable optimization and no evidence of significant overfitting.

Figure 4.10 shows the evolution of the total VAE loss for Dataset 2. Since the AE

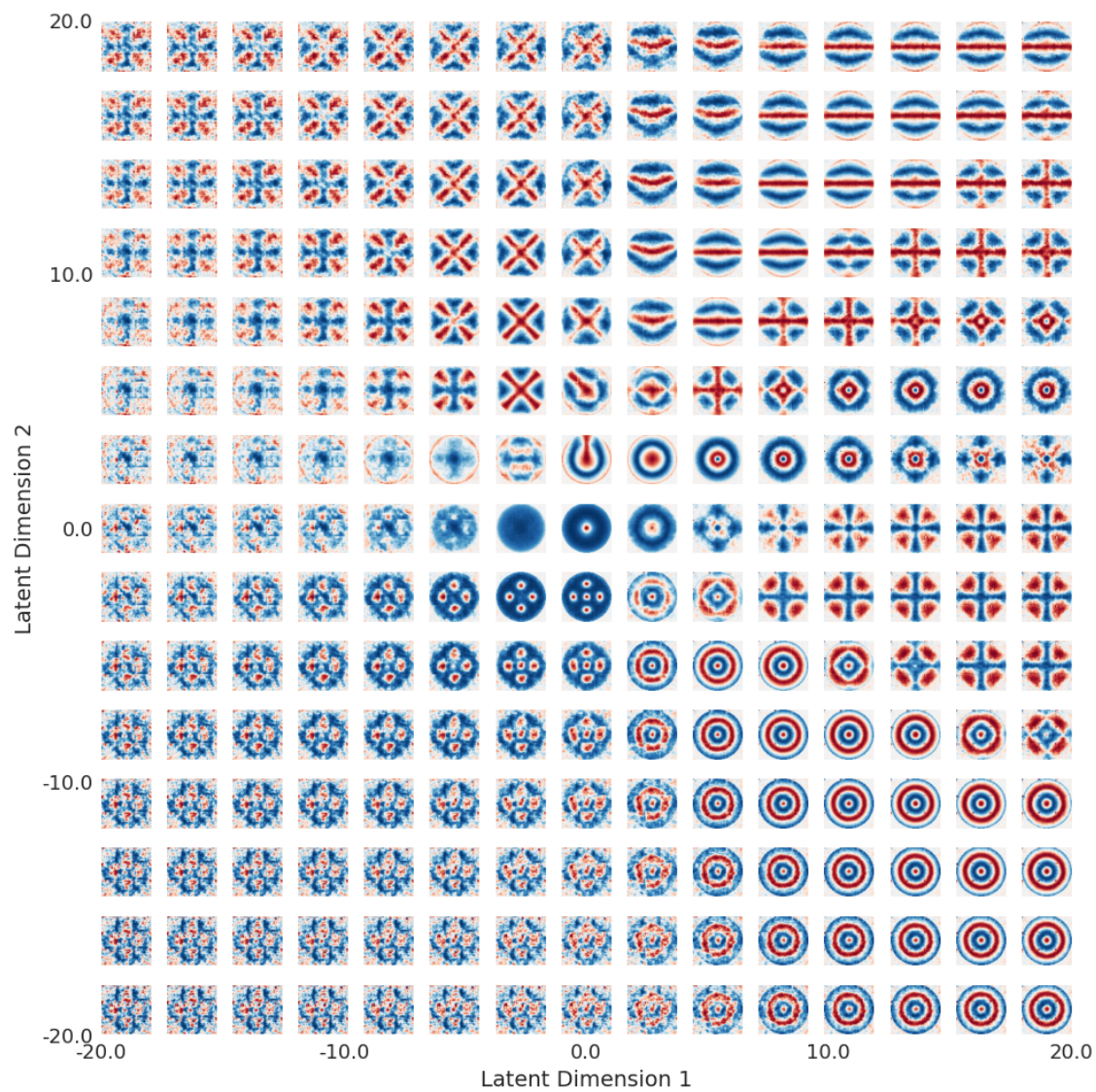


Figure 4.8: Caption

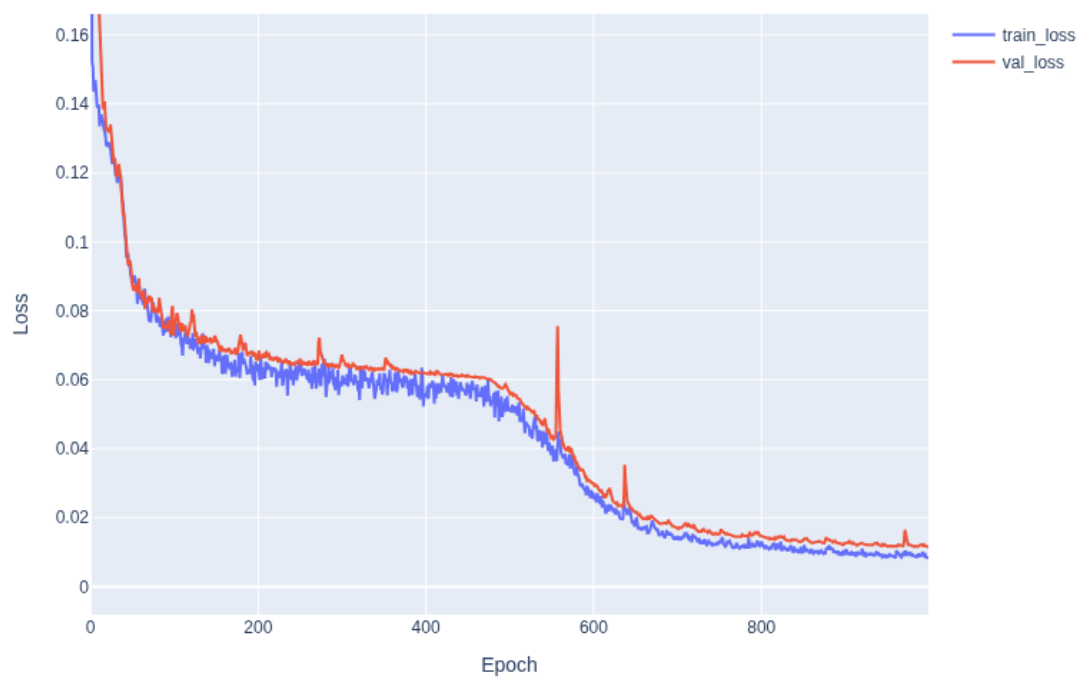


Figure 4.9: AE Loss vs Epoch, Dataset 2

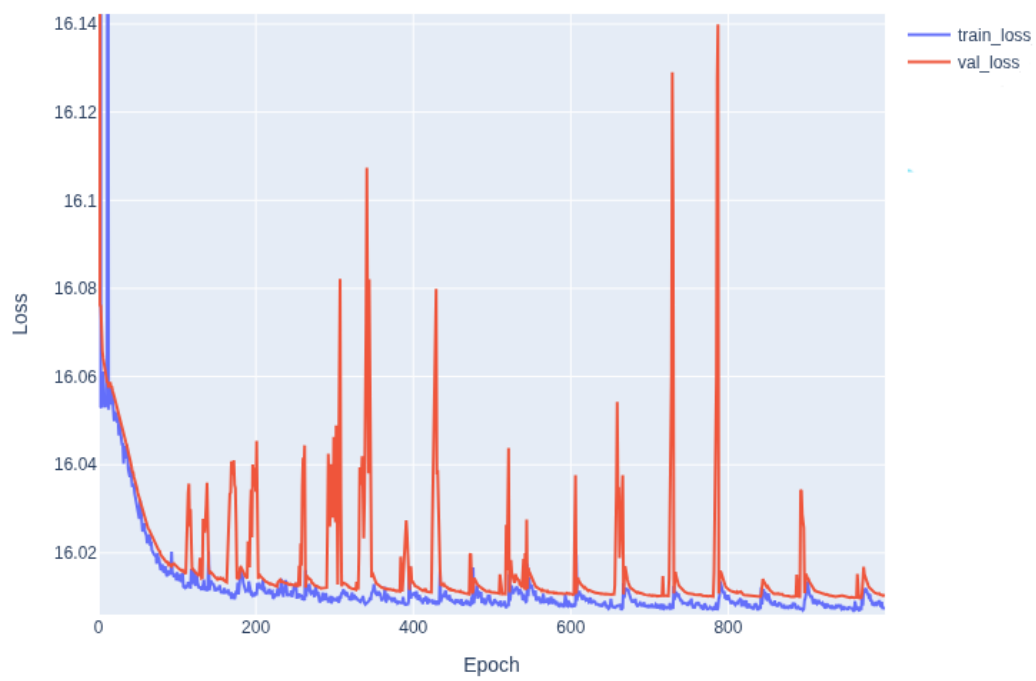
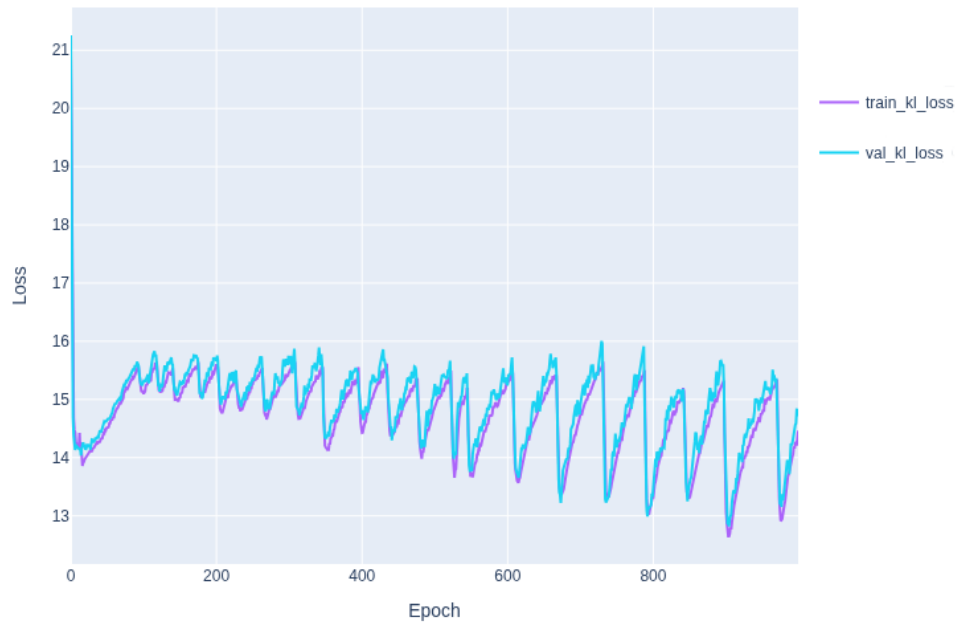


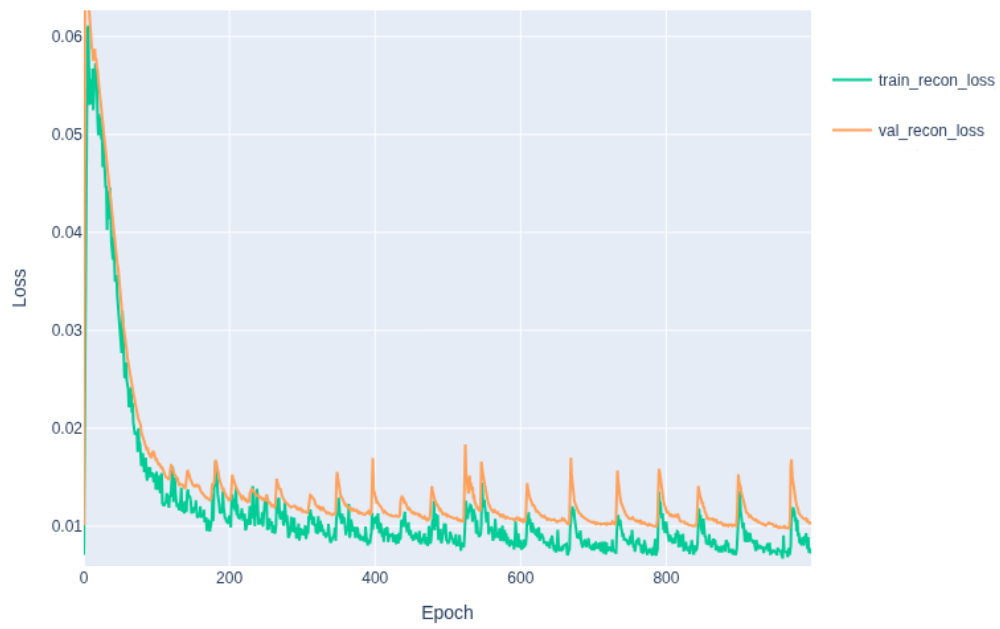
Figure 4.10: VAE Loss vs Epoch, Dataset 2

model and VAE model share a major portion of the architecture, in order to hasten the training process, the weights and biases of the trained AE model layers were fed into the VAE model for training. As observed previously, both training and validation losses decrease rapidly during the initial epochs, faster than those of the AE model, likely due to the weight initialization. After this early phase, the loss decreases more gradually and approaches a stable baseline.

The training curve remains smooth throughout optimization, while the validation loss exhibits intermittent spikes. The spikes are more pronounced in the case of the validation loss when compared to the training loss. The reasons for this are discussed in the previous section. The absence of a widening gap between the two curves suggests that overfitting is not significant despite the extended training duration.



(a) MSE Loss



(b) KL Divergence

Figure 4.11: VAE Loss Decomposition vs Epoch, Dataset 2

Figures 4.11a and 4.11b show the evolution of the Kullback–Leibler (KL) divergence and reconstruction loss terms during training for Dataset 2. The reconstruction loss decreases rapidly during the initial epochs and continues to decline gradually before stabilizing at a low value. Training and validation reconstruction curves closely track each other, indicating stable generalization. Although intermittent spikes appear in the validation curve, they remain transient and do not alter the overall convergence trend.

The KL divergence term exhibits a pronounced oscillatory pattern throughout training. This behavior arises from the applied KL thresholding strategy, which constrains the KL contribution to remain above a predefined minimum value. When the KL term approaches this threshold, the optimization alternates between enforcing regularization and improving reconstruction accuracy, leading to periodic fluctuations. The synchronized oscillations in both training and validation curves confirm that this effect is methodological rather than indicative of overfitting or instability.

Overall, the loss decomposition demonstrates that the variational objective maintains a controlled balance between reconstruction fidelity and latent space regularization.

Figure 4.12 compares the representative magnetization configurations from Dataset 2 with their corresponding reconstructions obtained from the AE and VAE. Since the original data are three-dimensional magnetization fields, the visualizations shown correspond to the central slice of the sample taken perpendicular to the z -axis. This slice is representative of the internal magnetic structure while allowing clear two-dimensional visualization of in-plane vectors and the out-of-plane component.

Both models accurately reproduce the dominant magnetic structures present in the central layer. The spatial organization of the magnetization patterns is preserved in both reconstructions. The AE output exhibits slightly sharper local features, while the VAE reconstructions appear marginally smoother due to the latent space regularization imposed by the KL term. For nearly uniform configurations, both models yield almost identical reconstructions. In more complex configurations with fine spatial variation, the VAE slightly suppresses high-frequency fluctuations but preserves the essential topology and spatial localization of magnetic structures.

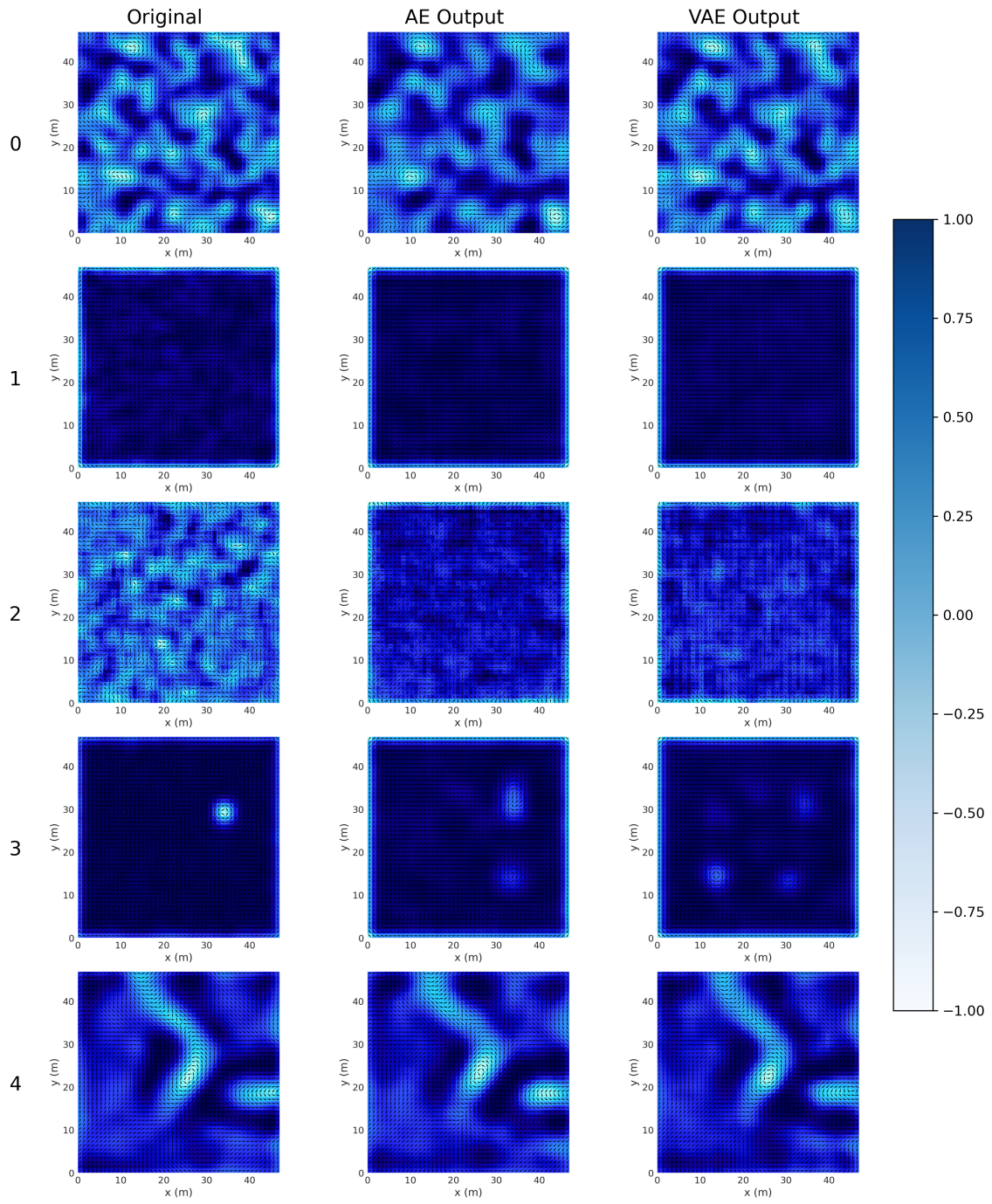


Figure 4.12: Comparison of original and reconstructed field images, Dataset 2

4.2.2 Clustering Evaluation

To assess clustering performance quantitatively, 10% of the dataset was manually labeled and used as a reference set. This labeled subset provides ground-truth class assignments for evaluating the correspondence between unsupervised cluster labels and visually identified micromagnetic states.

The rows in 4.12, as labeled, are randomly picked examples from each class. A good way to visually inspect the clustering quality is by plotting the clusters in the parameter space, as can be seen in 4.13.

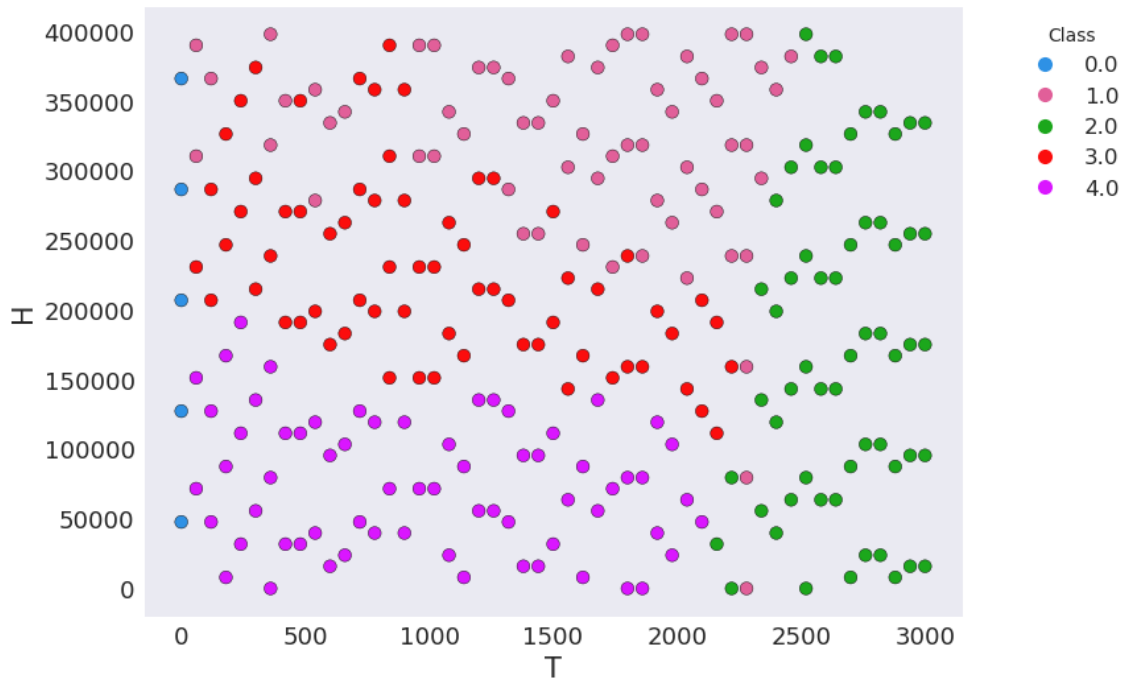


Figure 4.13: Labels plotted as a function of the applied external magnetic field on the horizontal axis and the temperature on the vertical axis.

Clustering in latent space was performed using a Gaussian Mixture Model (GMM) for both the AE and VAE representations. For the VAE latent space, the GMM was configured with $n_{\text{components}} = 5$ and a diagonal covariance structure. For the AE latent space, the same diagonal covariance type was used; however, $n_{\text{components}} = 6$ yielded better clustering

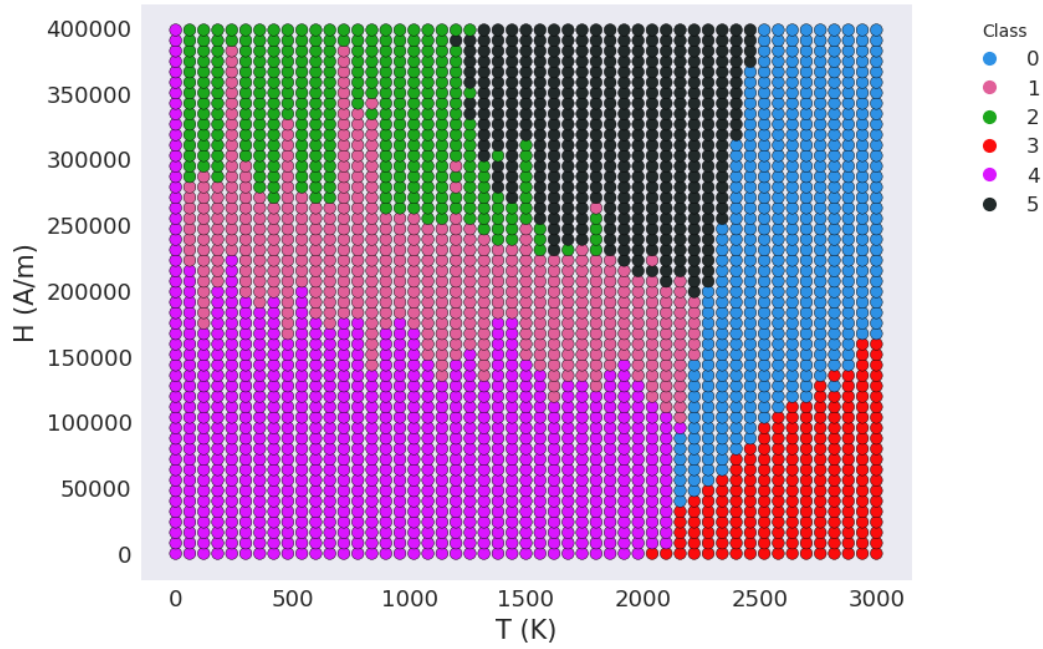
performance despite the presence of only 5 physical classes.

In a GMM with diagonal covariance matrices, each Gaussian component is allowed to have a distinct variance along each latent dimension, but correlations between dimensions are assumed to be zero. This means that the covariance matrix of each component contains non-zero entries only along the diagonal. Geometrically, each cluster is represented by an axis-aligned ellipsoid in latent space. This reduces the number of parameters compared to a full covariance model and improves numerical stability, particularly in higher-dimensional embeddings, while still allowing anisotropic cluster shapes.

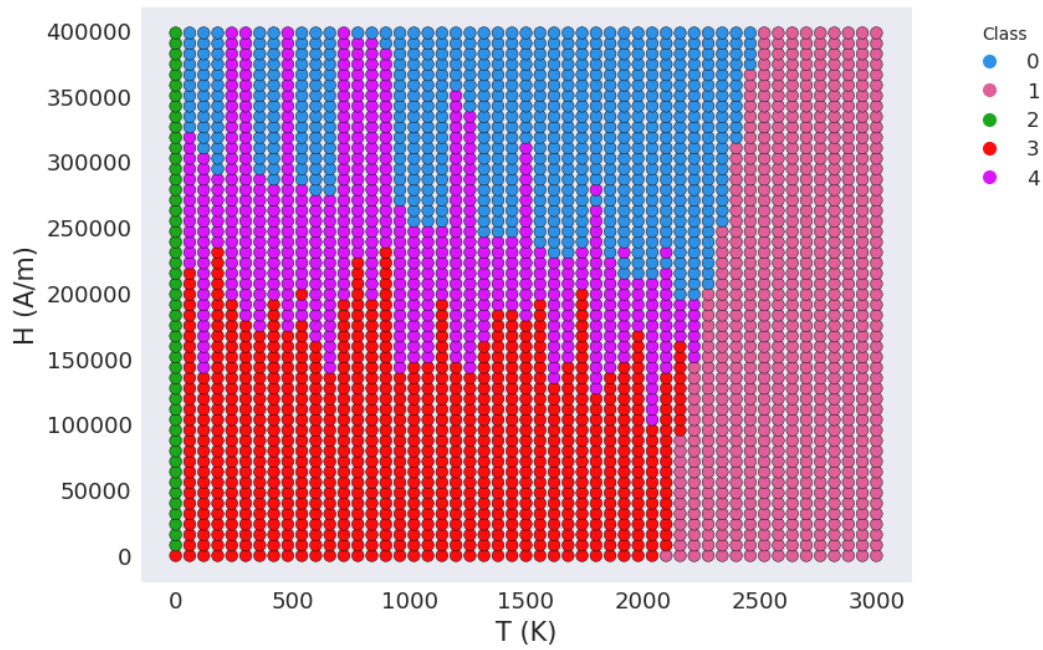
Figures 4.14a and 4.14b compare the clustering results obtained from the AE and the VAE plotted in the parameter space. The AE-based clustering reproduces the overall phase structure with high fidelity, capturing the dominant low-field, intermediate-field, and high-field regions. However, it fails to identify class 2, on the extreme left of the parameter space plot. It also splits class 1 on the extreme right into two, although examples from both the clusters are visually identical. Minor boundary irregularities and small misclassified pockets are visible near phase transition regions, where the magnetization configurations change more gradually.

The VAE-based clustering does a better job of recovering the global phase topology and preserves the principal phase boundaries. Compared to the AE, the VAE shows slightly smoother transitions between neighboring regions, consistent with the regularizing effect of the KL divergence.

Overall, both latent representations successfully recover the major phase structure present in the ground truth. The agreement between unsupervised clustering and manual labeling demonstrates that the learned latent spaces encode physically meaningful information about the magnetic phase behavior across the parameter range. Ultimately, the VAE models the data better, which is also reflected in the external metrics presented in 4.1



(a) AE clusters plotted in parameter space



(b) VAE clusters plotted in parameter space

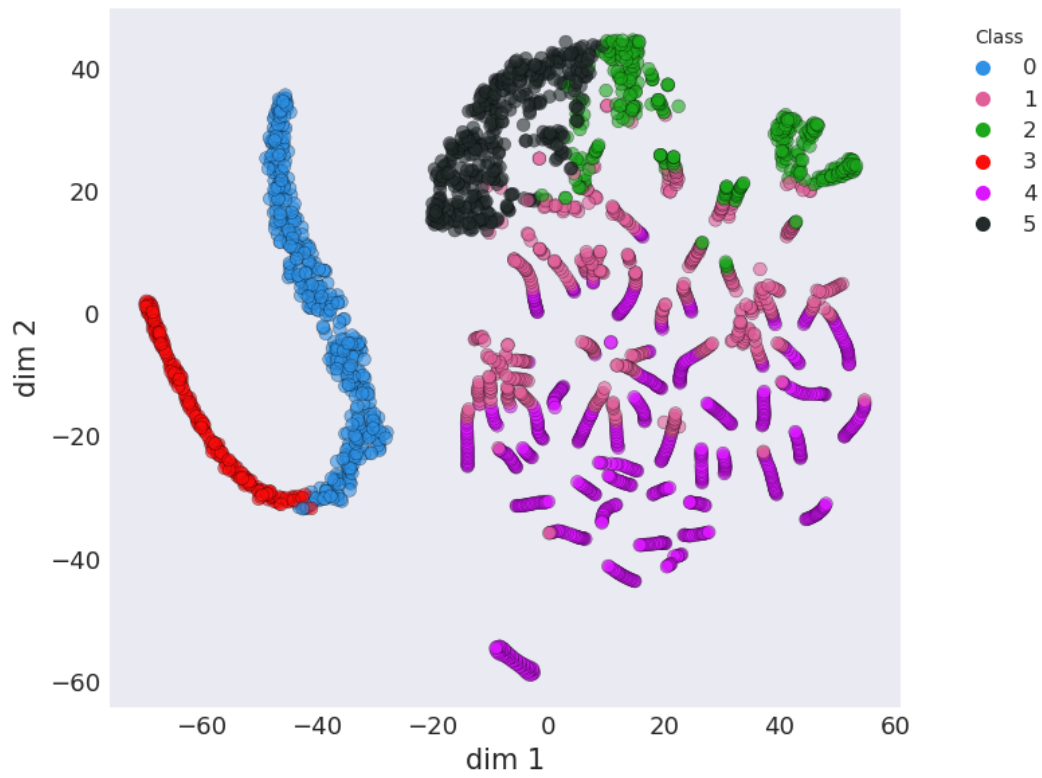
Figure 4.14: Results of GMM clustering

Table 4.1: Clustering performance comparison between AE and VAE latent representations for Dataset 2.

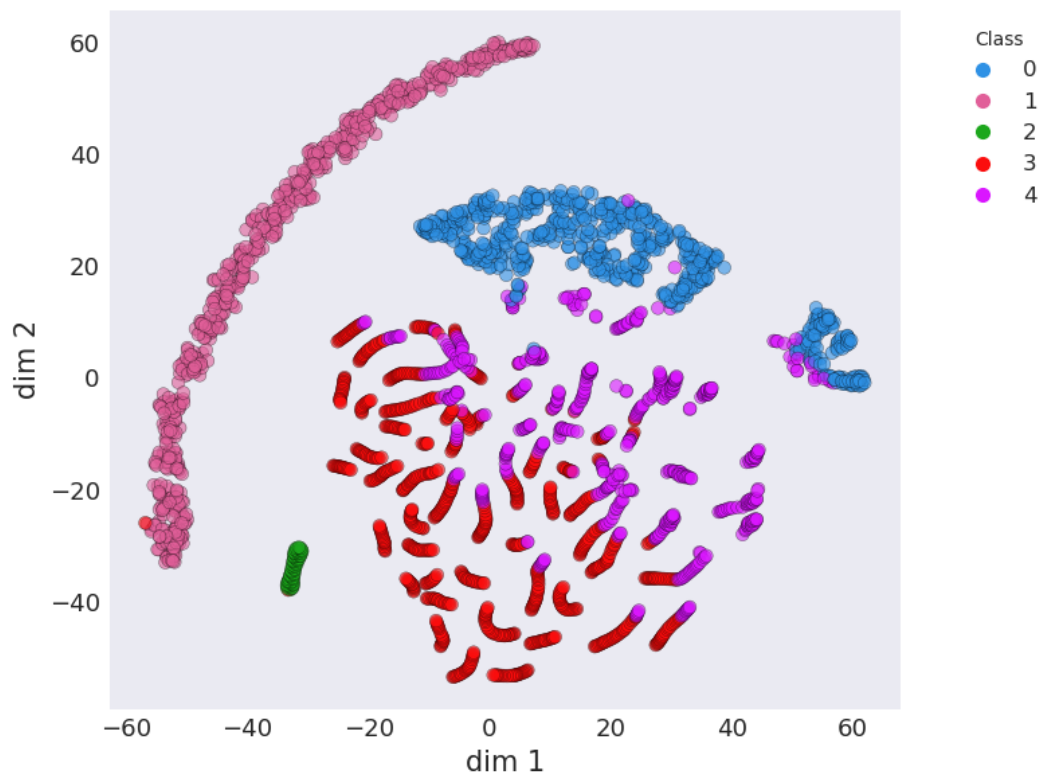
Model	Adjusted Rand Index (ARI)	Purity
AE	0.6230	0.8812
VAE	0.8093	0.9234

4.2.3 Latent Space Structure

Visualizing the latent spaces generated by the AE and VAE models for dataset 2 is not as simple as is the case for dataset 1. In Dataset 1, clustering and structure could be directly visualized in a two-dimensional latent space, allowing straightforward geometric interpretation of cluster boundaries and transitions. In contrast, Dataset 2 is embedded in an eight-dimensional latent space. Any visual representation, therefore, requires a nonlinear projection from the high-dimensional latent space into two dimensions. Such projections inevitably introduce distortions, potentially obscuring true distances, densities, or cluster separations. As a result, quantitative metrics such as ARI and purity play a more critical role. Visualizations, like the one presented below, are less reliable.



(a) AE projection



(b) VAE projection

Figure 4.15: t-SNE projections of the latent spaces with GMM-assigned classes

To visualize the eight-dimensional latent space, t-distributed Stochastic Neighbor Embedding (t-SNE) was employed. t-SNE is a nonlinear dimensionality reduction technique designed to preserve local neighborhood structure[32].

Figure 4.15 shows two-dimensional t-SNE projections of the latent spaces learned by the AE (top) and VAE (bottom). Each point corresponds to a magnetization configuration, colored according to its assigned class.

Both the latent space projections display elongated structures. In retrospect, this supports the choice of diagonal covariance for the GMM for the reasons stated above. In both projections, several clusters form partially overlapping structures. But this overlap can arise from either the latent space structure or the projection itself. Clusters in the VAE projection display less fragmentation when compared to its deterministic counterpart, indicating lower intra-class variability.

4.2.4 Generative Exploration of the Latent Space

In order to explore the latent space, The uniform sampling approach taken in subsection 4.1.4 is not feasible for this dataset. Since the latent space is eight-dimensional, sampling from it is ineffective. Hence, an alternative approach is to start at a sample from the dataset and explore how the model reconstruction looks when its latent codes are perturbed by controlled amounts of noise. Gaussian noise is added to the latent vectors with varying values for the standard deviation, σ . This allows us to explore the latent space, beginning close to an actual representation and progressively moving farther away. The sample chosen corresponds to the example illustrated in the third row (class 2) of Figure 4.12. For visualization, the reconstructed tensor is reduced to a two-dimensional mid-plane slice along the z -direction as before.

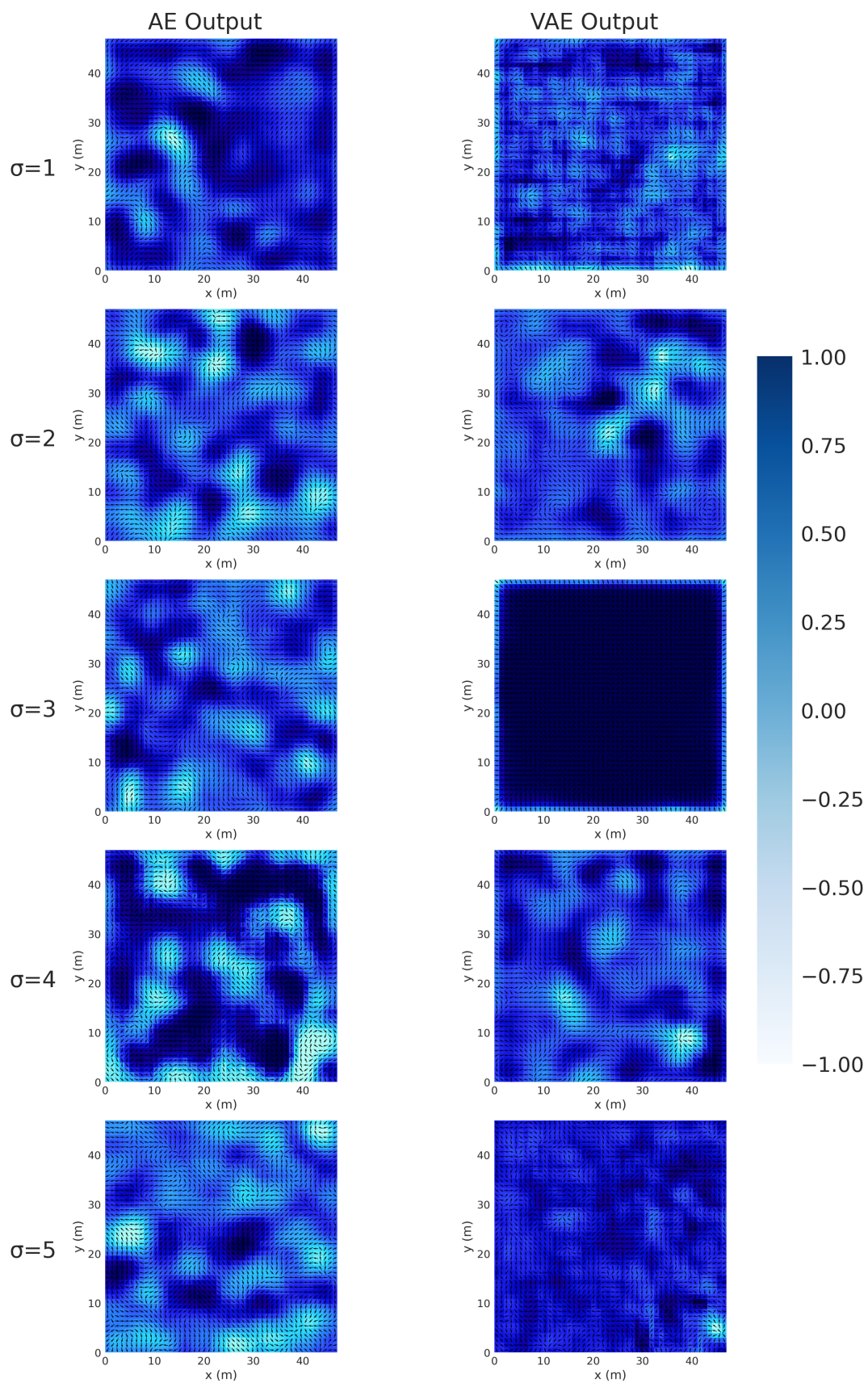


Figure 4.16: Perturbed input for Latent space exploration

Figure 4.16 compares reconstructed center-plane slices from the AE and VAE) under increasing latent-space perturbations. In both models, the decoded fields remain physically plausible across the full noise range, and the overall structural quality of the reconstructions is comparable. The primary difference lies not in reconstruction fidelity but in the extent of variability explored as the noise amplitude increases. The AE produces structured and distinct configurations, but the evolution across noise levels appears more localized in latent space, as the configurations would all be classified under the same class. In contrast, the VAE exhibits a broader diversity of classes. The reconstructed fields span a wider range of configurations while preserving global coherence, suggesting that the latent space supports smoother and more extensive transitions in the latent space.

The efficacy of the standard AE at this task, although surprising, might be explained by the presence of noise in the dataset. The AE is exposed to stochastic variability during training and implicitly acts as Denoising autoencoder[33]. The regularizing effect of denoising autoencoders could explain this.

4.3 Outlook

This work demonstrates that combining Machine learning-based dimensionality reduction with unsupervised clustering is a practical and effective strategy for analyzing high-dimensional micromagnetic data. Convolutional autoencoders successfully learned compact latent representations that preserved the essential structure of magnetization configurations, substantially alleviating the curse of dimensionality. Variational autoencoders further improved the organization of the latent space by imposing a probabilistic structure. The learned embeddings showed meaningful grouping behavior across datasets with different geometries and physical parameters, indicating that the framework generalizes beyond a single system.

A promising direction for future work is the use of Gaussian Mixture Variational Autoencoders (GMVAEs)[9]. Unlike standard VAEs, which impose a unimodal prior, GMVAEs employ a mixture-of-Gaussians prior that can explicitly model multimodal latent distributions. This is especially attractive for clustering tasks, as discrete modes in la-

tent space can correspond directly to distinct physical phases or configuration classes. By integrating dimensionality reduction and clustering into a single generative framework, GMVAEs may reduce the need for a separate clustering step and yield more physically meaningful partitions.

Experiments conducted on another dataset not included in this thesis suggest that alternative loss functions may address some of these shortcomings. In particular, perceptual loss — based on feature-space similarity rather than pixel-wise differences — proved highly effective at preserving large-scale structural characteristics while remaining tolerant to small local variations. This indicates that incorporating domain-aware or feature-based reconstruction objectives could substantially improve representation quality for complex physical data.

Additionally, experiments on a separate dataset not included in this study suggest that perceptual loss functions — such as those based on deep feature similarity[34] — can significantly improve the preservation of large-scale structure, suggesting a promising direction for enhancing reconstruction quality in complex physical systems.

In summary, the results indicate that deep unsupervised learning provides a powerful toolkit for high-dimensional data analysis, but further advances in model design, loss functions, and probabilistic structure are likely to yield even more robust and physically informative outcomes.

4.4 Software Availability

The software developed for this study is available as open-source code at https://github.com/Sassy-Buoy/3D_periodic_clustering and <https://github.com/Sassy-Buoy/hyper-param-optim-4-vector-field-clustering>. The repositories contain the implementation of the autoencoder and variational autoencoder models, data preprocessing pipelines, clustering workflows, and scripts required to reproduce the experiments and figures presented in this thesis. Documentation is provided in the repository’s README file.

Bibliography

- [1] Charu C Aggarwal, Alexander Hinneburg, and Daniel A Keim. On the surprising behavior of distance metrics in high dimensional space. In *International conference on database theory*, pages 420–434. Springer, 2001.
- [2] Naomi Altman and Martin Krzywinski. The curse (s) of dimensionality. *Nat Methods*, 15(6):399–400, 2018.
- [3] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. Technical report, Stanford, 2006.
- [4] Marijan Beg, Martin Lang, and Hans Fangohr. Ubermag: toward more effective micromagnetic workflows. *IEEE Transactions on Magnetics*, 58(2):1–5, 2021.
- [5] Kevin Beyer, Jonathan Goldstein, Raghu Ramakrishnan, and Uri Shaft. When is “nearest neighbor” meaningful? In *International conference on database theory*, pages 217–235. Springer, 1999.
- [6] William Fuller Brown. *Micromagnetics*. Interscience Publishers, New York, 1963.
- [7] Caiyq and Ijin08. Neuralmag: Initial release v1.0.0, 2024.
- [8] John MD Coey. *Magnetism and magnetic materials*. Cambridge university press, 2010.
- [9] Nat Dilokthanakul, Pedro AM Mediano, Marta Garnelo, Matthew CH Lee, Hugh Salimbeni, Kai Arulkumaran, and Murray Shanahan. Deep unsupervised clustering

- with gaussian mixture variational autoencoders. *arXiv preprint arXiv:1611.02648*, 2016.
- [10] Michael Joseph Donahue and Donald G Porter. Oommf user’s guide: Version 1.0, 1999.
 - [11] William Falcon and The PyTorch Lightning team. PyTorch Lightning, March 2019.
 - [12] Albert Fert, Vincent Cros, and Joao Sampaio. Skyrmions on the track. *Nature nanotechnology*, 8(3):152–156, 2013.
 - [13] Aurélien Géron. *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow*. ” O’Reilly Media, Inc.”, 2022.
 - [14] D Hendrycks. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016.
 - [15] Geoffrey E Hinton and Ruslan R Salakhutdinov. Reducing the dimensionality of data with neural networks. *science*, 313(5786):504–507, 2006.
 - [16] Zhexue Huang. Extensions to the k-means algorithm for clustering large data sets with categorical values. *Data mining and knowledge discovery*, 2(3):283–304, 1998.
 - [17] Alex Hubert and Rudolf Schäfer. *Magnetic domains: the analysis of magnetic microstructures*. Springer Science & Business Media, 1998.
 - [18] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.
 - [19] Plotly Technologies Inc. Collaborative data science, 2015.
 - [20] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456. pmlr, 2015.
 - [21] Md Amirul Islam, Matthew Kowal, Sen Jia, Konstantinos G Derpanis, and Neil DB Bruce. Global pooling, more than meets the eye: Position information is encoded

- channel-wise in cnns. In *Proceedings of the IEEE/CVF International conference on computer vision*, pages 793–801, 2021.
- [22] Leonard Kaufman and Peter J Rousseeuw. *Finding groups in data: an introduction to cluster analysis*. John Wiley & Sons, 2009.
 - [23] Diederik P Kingma. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
 - [24] Diederik P Kingma, Max Welling, et al. Auto-encoding variational bayes, 2013.
 - [25] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035, 2019.
 - [26] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
 - [27] LKJP Rousseeun and P Kaufman. Clustering by means of medoids. In *Proceedings of the statistical data analysis based on the L1 norm conference, neuchatel, switzerland*, volume 31, page 28, 1987.
 - [28] Frederic Ros and Rabia Riad. *Feature and Dimensionality Reduction for Clustering with Deep Learning*, pages 11–14. Springer, 2024.
 - [29] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.
 - [30] Mayu Sakurada and Takehisa Yairi. Anomaly detection using autoencoders with non-linear dimensionality reduction. In *Proceedings of the MLSDA 2014 2nd workshop on machine learning for sensory data analysis*, pages 4–11, 2014.

- [31] Erich Schubert, Jörg Sander, Martin Ester, Hans Peter Kriegel, and Xiaowei Xu. Dbscan revisited, revisited: why and how you should (still) use dbscan. *ACM Transactions on Database Systems (TODS)*, 42(3):1–21, 2017.
- [32] Arne Vansteenkiste, Jonathan Leliaert, Mykola Dvornik, Mathias Helsen, Felipe Garcia-Sanchez, and Bartel Van Waeyenberge. The design and verification of mu-max3. *AIP advances*, 4(10), 2014.
- [33] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th international conference on Machine learning*, pages 1096–1103, 2008.
- [34] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.
- [35] Arthur Zimek, Erich Schubert, and Hans-Peter Kriegel. A survey on unsupervised outlier detection in high-dimensional numerical data. *Statistical Analysis and Data Mining: The ASA Data Science Journal*, 5(5):363–387, 2012.

Acknowledgements

I would like to express my sincere gratitude to Swapneel Amit Pathak for his constant guidance, insightful ideas, and patient support throughout this work. His mentorship and hands-on assistance were invaluable, and this thesis would not have been possible without him.

I would like to thank Prof. Hans Fangohr for his supervision, for the opportunity to work on this project, and for his constructive feedback and helpful ideas.

I am thankful to Samuel Holt for generating the datasets used in this study and for providing many interesting ideas and discussions that shaped the direction of the project.

I am grateful to Dr. Guido Meier for his valuable feedback and for helping me improve my presentation skills.

Statutory Declaration

I declare that I have authored the Bachelor's/Master's thesis independently and have not used any aids other than those specified. I have clearly labelled all passages that are taken from other works in terms of wording or meaning as borrowed material, stating the exact source in each case. I confirm that I have not already submitted this thesis in the same or a similar version in another examination procedure. If electronic aids based on generative artificial intelligence (gAI) were used in the course of the preparation of this Bachelor's/Master's thesis, I confirm that my own work was the main focus and that complete documentation of all aids used is available in accordance with good scientific practice. I am responsible for any incorrect or distorted content, incorrect references, violations of data protection and copyright law or plagiarism generated by the gAI. I agree that the Master's thesis may be published.

Hamburg, February 26, 2026

A handwritten signature in cursive script, appearing to read 'Sasi', with a horizontal line drawn underneath it.